计算理论复习讲义

图灵机模型 · 可计算性理论 · 计算复杂性理论

这本讲义解决什么问题？

计算理论的课件往往只放定义和定理骨架，跨度大、省略多。本讲义将课程内容重新梳理为**人类易读的逻辑链条**：

1. **把抽象定义翻译成人话**：每个核心概念先讲“为什么要这样定义”，再给形式化符号。
2. **把证明写成可复现的模板**：对角化、映射归约、Rice 定理、NP 完全性归约均拆解为固定的“分步答题套路”。
3. **把两份作业全部做成闭环参考**：5 道图灵机设计题**全部给出完整转移表**（均通过脚本穷举验证），9 道可计算与复杂性证明题给出可直接书写的规范证明。

全书结构导览

- **第 1 节（课程全景）**：用一张主线图串起“计算模型 → 可计算性 → 计算复杂性”三大模块。
- **第 2 节（图灵机）**：七元组、瞬时描述 (ID)、单带手写编程套路、变形机器等价性与通用图灵机。
- **第 3–4 节（可计算性）**：可判定 vs. 可识别、封闭性证明模板、停机问题、映射归约方向、Rice 定理与 PCP。
- **第 5–7 节（计算复杂性）**：P 与 NP、Cook 定理、六大经典 NP 完全归约链、PSPACE/QBF/L/NL。
- **第 8–10 节（实战与习题全解）**：证明题排雷清单、5 道图灵机习题完整转移表、9 道理论习题完整证明。
- **第 11–12 节（速查与自测）**：原课件页码索引、18 道考前遮答自测题。

资料来源说明

依据本目录 4 份课件（《00-intro-slides》《07-图灵机》《08-图灵机和可计算》《12-计算复杂性》）与 2 份配套习题（《图灵机设计习题》《可计算和复杂性理论习题》）编写。本讲义供课后复习与理解使用，不代表教师公布的考纲或标准答案。

目录

1 先把整门课的主线串起来 **3**

1.1 为什么计算理论里总把“问题”叫做“语言”? . . . 3

1.2 输入长度与数值大小：二进制和一元编码差在哪里? . . . 3

2 图灵机：状态、纸带与手写编程方法 **3**

2.1 用人话读懂确定图灵机（DTM）的七元组 . . . 4

2.2 瞬时描述（ID）：怎么在纸上拍下计算快照? . . . 4

2.3 接受、拒绝与死循环：识别器和判定器的根本分界 . . . 4

2.4 手写单带图灵机的“三板斧”与正确性写法 . . . 5

2.5 图灵机的变形、编码与通用图灵机 . . . 5

3 可判定、可识别与枚举器 **6**

3.1 一张表分清“可判定（递归）”与“可识别（递归可枚举）” . . . 6

3.2 核心等价定理：L 可判定 $\iff$ L 与 $\overline{L}$ 都可识别 . . . 6

3.3 封闭性汇总与“串行 vs. 交错”证明模板 . . . 7

3.4 枚举图灵机（Enumerator）：换一种角度看可识别与可判定 . . . 7

3.5 基数论证：为什么世界上绝大多数语言都无法被图灵机识别? . . . 8

4 不可判定性：对角化、映射归约与 Rice 定理 **8**

4.1 三大锚点问题：L_d、A_{TM}（或 L_{ACC}）与 H_{TM}（或 L_{HALT}） . . . 8

4.2 映射归约 $A \leq_m B$：方向怎么记才永远不会反? . . . 9

4.3 构造辅助机器 N 的核心秘诀：“写代码”不等于“跑代码”! . . . 9

4.4 四个经典不可判定/不可识别问题的归约表 . . . 10

4.5 Rice 定理：一秒钟秒杀所有“语言性质”不可判定题 . . . 10

4.6 图灵归约 $\leq_T$、波斯特对应问题（PCP）与王氏贴砖 . . . 11

5 从可计算性转向计算复杂性：P、NP 与 NP 完全 **12**

5.1 时间与空间复杂性的度量规则（最坏输入 + 最坏分支） . . . 12

5.2 理解 NP 的最佳方式：多项式时间验证器（Verifier） . . . 13

5.3 判定问题 vs. 优化问题，以及 NP 完全性（NPC）的标准四步写法 . . . 13

6 Cook 定理与经典 NP 完全归约全景图 **14**

6.1 Cook 定理（$\forall A \in \mathrm{NP},\ A \leq_p \mathrm{SAT}$）：怎样用布尔公式模拟图灵机? . . . 14

6.2 SAT $\leq_p$ CNF-SAT $\leq_p$ 3SAT：核心是“保可满足性（Equisatisfiable）” . . . 15

6.3 3SAT $\leq_p$ 独立集（IS）：怎样把逻辑公式翻译成一张图? . . . 16

6.4 独立集（IS）、团（CLIQUE）与顶点覆盖（VC）的三角铁关系 . . . 16

6.5　顶点覆盖（VC）$\leq_p$ 支配集（DS）：在每条边上架一个“三角形哨兵” 16
6.6　哈密顿环（HAM-CYCLE）$\leq_p$ 旅行商问题（TSP）：经典的 1/2 边权赋权法 17
6.7　课件其余 NP 完全问题速览表 17

7　空间复杂性、博弈问题与对数空间类　17
7.1　复杂性类大一统层级与 Savitch 定理 17
7.2　PSPACE 完全问题：带量词布尔公式（QBF）与双人博弈 18
7.3　P 完全与对数空间类（L 与 NL）：为什么这里必须换用“对数空间归约 $\leq_L$”? 18

8　考场证明题“防扣分”十条军规　19

9　图灵机设计习题：5 道题完整转移表与详解　20
9.1　题 1：设计单带确定图灵机识别 $\{0^n1^n2^n \mid n \geq 0\}$，并写出 001122 的 ID 序列 20
9.2　题 2：识别 $\{w \in \{0,1\}^* \mid w$ 中0 和1 数目相同$\}$，并写出 001010 的 ID 序列 21
9.3　题 3：设计单带确定图灵机识别 $\{0^{2^n} \mid n \geq 0\}$ 23
9.4　题 4：设计单带确定图灵机识别 $\{w\#w \mid w \in \{0,1\}^*\}$ 23
9.5　题 5：设计单带确定图灵机识别 $\{ww \mid w \in \{0,1\}^*\}$ 24

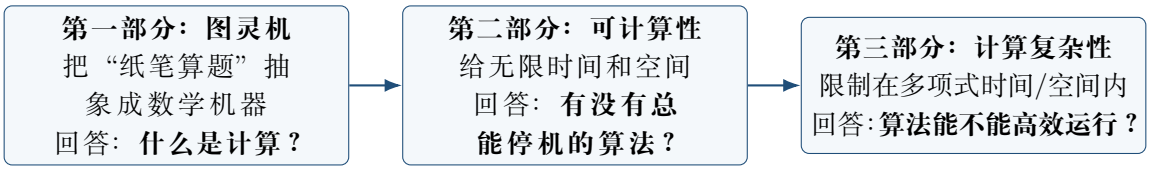
10　可计算与复杂性理论习题：9 道题规范证明全解　26
10.1　题 1：证明递归语言在并操作下封闭（若 L, L' 均递归，则 $L \cup L'$ 也递归） 26
10.2　题 2：证明递归可枚举语言在并操作下封闭（若 L, L' 均递归可枚举，则 $L \cup L'$ 也递归可枚举） 26
10.3　题 3：证明集合 $S = \{\langle M\rangle \mid L(M) = \{ww^R \mid w \in \{0,1\}^*\}\}$ 是不可判定的 26
10.4　题 4：证明集合 $T = \{\langle M\rangle \mid M$ 在所有输入上均停机接受$\}$ 是不可判定的 27
10.5　题 5：证明最大团（Clique）问题是 NP 完全问题 28
10.6　题 6：证明若 A 是 NP 完全问题且有多项式时间算法，则 P = NP 28
10.7　题 7：将下列两个布尔表达式转换为保持可满足性一致的 3CNF 形式 29
10.8　题 8：证明 4TA-SAT（至少有 4 个不同满足赋值）是 NP 完全问题 29
10.9　题 9：证明半团问题（Half-Clique）是 NP 完全问题 30

11　原课件速查索引与考前自测 18 问　32
11.1　遇到不懂的概念，去原课件哪一页查? 32
11.2　考前最后自测 18 问（左边遮住右边，能脱口而出就稳了） 32

1　先把整门课的主线串起来

【课件对应】《00-intro-slides》PDF 第 1–5 页；《12-计算复杂性》PDF 第 1 页。（注：本文“PDF 第 x 页”均指文件实际页码，每页通常含 6 张幻灯片。）

整门课其实只追问递进的三个问题：**怎样用数学语言定义“计算”？哪些问题根本不存在算法？存在算法的问题需要消耗多少时间和空间？**



1.1　为什么计算理论里总把“问题”叫做“语言”？

平时我们说“判断图 G 里有没有大小为 k 的团”，输入是一个图和一个整数。但图灵机的纸带上只能写字符串。只要把输入实例按固定规则编码成字符串 $\langle G, k\rangle$，再把所有**答案为“是”的实例编码**收集成一个集合：

$$\text{CLIQUE} = \{\langle G, k\rangle \mid G \text{ 中存在大小至少为} k \text{ 的团}\},$$

这个字符串集合就叫做一个**语言（Language）**。

【直观理解】判定问题 = 集合成员判断　把“求解判定问题”翻译成“判断字符串 x 是否属于集合 L”，最大的好处是可以用集合运算（并、交、补）和函数映射（归约）来精确研究问题的难度。记住一句话：**语言是问题（实例集合），图灵机是程序；一个语言可以有无数台不同的图灵机来识别，但一台确定的图灵机只对应唯一一个接受语言 $L(M)$。**

1.2　输入长度与数值大小：二进制和一元编码差在哪里？

在衡量算法快慢时，复杂性理论总是以**输入字符串的长度** $n = |x|$ 为基准，而不是以输入所代表的数值 N 为基准：

- **二进制（或十进制等位值制）**：正整数 N 写出来只需要 $n = \Theta(\log N)$ 位。反过来，长度为 n 的二进制串代表的数值高达 $N = \Theta(2^n)$。
- **一元编码（如罗马数字式的记数）**：用 N 个 1 表示整数 N，长度 $n = N$，极其冗长。

【易错警示】为什么耗时 $O(N)$ 的算法不一定是多项式时间算法？　如果输入是一个二进制整数 N，它的输入长度是 $n = \lfloor \log_2 N \rfloor + 1$。一个运行 N 步的算法，按输入长度 n 来算是 $\Theta(2^n)$ 步，属于**指数时间**（也叫**伪多项式时间**）。后面讲子集和问题（SUBSET-SUM）的 $O(nt)$ 动态规划为什么不违背 NP 完全性，核心就在这里。

2　图灵机：状态、纸带与手写编程方法

【课件对应】《07-图灵机》PDF 第 1–4 页（幻灯片 3–23）。

2.1　用人话读懂确定图灵机（DTM）的七元组

图灵机就是把人用纸笔列竖式算加法的动作剥离到最简：一条无限长的格子纸带（草稿纸）、一个一次只看一格的读写头（眼睛和笔）、以及一个记住当前做到哪一步的有限状态控制器（脑子里的短期记忆）。

课件将确定图灵机记为七元组 $M=(Q,\Sigma,\Gamma,\delta,q_0,B,F)$：

符号	名称	直观含义与做题注意点
Q	有穷状态集	机器的“步骤标签”，数量固定有限，**绝对不能用来存无限增长的计数器** n。
Σ	输入字母表	初始允许出现在输入串里的字符集（不含空白符 B）。
Γ	纸带字母表	纸带上允许出现的所有字符，$\Sigma\subseteq\Gamma$，通常还会加入标记字符（如 X,Y,Z）。
δ	转移函数	机器的程序规则：$\delta(q,a)=(p,b,D)$，其中 $q\in Q\setminus F$，$a,b\in\Gamma$，$D\in\{L,R\}$。
q_0	初始状态	计算开始时机器所处的状态，此时带头对准输入串的最左字符。
B	空白符	$B\in\Gamma\setminus\Sigma$，初始化时输入串以外的所有格子都填满 B。**切勿把空白符 B 和空串 ε 混为一谈**。
F	停机状态集	$F\subseteq Q$。做语言判定/识别题时，通常细分为接受状态 q_{yes}（或 q_a）和拒绝状态 q_{no}（或 q_r）。

一条规则 $\delta(q,0)=(p,X,R)$ 的意思非常直接：**当处于状态 q 且带头看到字符 0 时，把当前格改写成 X，带头向右移动一格，状态切换为 p**。

2.2　瞬时描述（ID）：怎么在纸上拍下计算快照？

为了在考卷上写出图灵机的运行轨迹，我们用字符串 uqv 表示某一时刻的完整配置，称为**瞬时描述（Instantaneous Description, ID）**：

> **瞬时描述（ID）的三条读写铁律**
>
> 在记号 $\alpha q\beta$（其中 $\alpha,\beta\in\Gamma^*$，$q\in Q$）中：
>
> 1. **状态写在被扫描字符的正前方**：带头此刻正指着 β 的**第一个字符**（右边那格），而不是左边那格。
> 2. **右移跨过右邻字符，左移把左邻字符抱到右边**：
>   - 若 $\delta(q,a)=(p,b,R)$，则 $ucqav\vdash ucbpv$（写下 b，状态 p 跳到右移后的新位置前）。
>   - 若 $\delta(q,a)=(p,b,L)$，则 $ucqav\vdash upcbv$（写下 b，状态 p 退到左邻字符 c 的前面）。
> 3. **边界补空白 B**：若带头在最左端左移，如 qav 执行 (p,b,L)，会踩进左边的空白格，写作 $pBbv$；若带头移到最右端空白上，写作 upB，**正在被带头扫描的 B 绝不能省略**。

符号 $\vdash$ 表示一步转移，$\vdash^*$ 表示经过 0 步或有限多步转移。

2.3　接受、拒绝与死循环：识别器和判定器的根本分界

当输入 w 放上纸带开始运行后，只可能出现三种结局：

1. 进入接受状态 q_a 并停机（**接受**）；

2. 进入拒绝状态 q_r，或找不到可用转移规则而卡停（**拒绝**）；
3. 永远在纸带上移动而不进入停机状态（**不停机 / 死循环**）。

由此引出可计算性理论最核心的一对概念：

- **M 识别（Recognize）语言 L**：$w \in L \iff M$ 在 w 上最终停机接受。至于 $w \notin L$ 时，M 可以停机拒绝，也**允许永远不停机**。对应语言称为**图灵可识别语言（递归可枚举语言，RE）**。
- **M 判定（Decide）语言 L**：M 对**所有输入** $w \in \Sigma^*$ 都必须有限步停机——$w \in L$ 时停于接受状态，$w \notin L$ 时停于拒绝状态。只有总能停机的图灵机才叫**算法**。对应语言称为**图灵可判定语言（递归语言，Recursive）**。

【易错警示】运行了一万步还没停，能说明 $w \notin L$ 吗？ 不能！对于一般的识别器，只要它还没停下来，你就无法区分它是“再跑两步就要接受了”还是“已经陷入死循环永远不接受”。

2.4　手写单带图灵机的“三板斧”与正确性写法

单带图灵机没有随机访问内存，手写转移表时最稳妥的设计模式是**“标记—扫描配对—回退左端”循环**：

1. **给每个状态起一句话功能名**：例如 s（在左端找下一个未处理的字符）、a（带着刚读到的信息向右找配对字符）、r（一路向左退回左端空白 B）、c（扫描检查尾部是否全部合法标记）。
2. **用不同标记符号区分阶段，绝不在中间乱写空白 B**：如果在纸带中间写 B，回退状态 r 寻找左端边界 B 时就会提前停错位置。应改用 X, Y, Z 等专用标记符。
3. **说明为什么正确（不变量 + 停机量）**：只画一条特定输入的运行轨迹只能算测试，不能算证明。规范说明只需三句话：（1）每轮循环保持什么对应关系（循环不变量）；（2）为什么只有合法串能通过最终检查；（3）每轮减少哪个有限非负整数（如未标记字符数），从而保证必停。

2.5　图灵机的变形、编码与通用图灵机

课件介绍了多种图灵机变体，结论非常统一：**它们识别语言的能力与标准单带确定图灵机完全等价（但运行时间可能相差很大）**。

变形模型	单带确定图灵机（DTM）如何模拟它？	模拟的时间代价 / 备注
单停机状态 TM (onestopTM)	将原机器所有接受状态合并/跳转到唯一停机状态 q_f，拒绝分支改为死循环或卡停	识别能力（RE）不变
多带图灵机（k-tape TM）	在一条纸带上分 $2k$ 条轨道（或用分隔符），同时记录 k 条带的内容与 k 个带头位置标记；模拟一步需来回扫描整条有效区	$T(n)$ 步多带机 $\to O(T(n)^2)$ 步单带机（**平方级代价**）
非确定图灵机（NTM）	$\delta(q,a)$ 返回有限个候选动作集合；只要**存在一条**计算分支进入接受状态就算接受。DTM 用**广度优先（按层）**遍历计算树来模拟	$T(n)$ 步 NTM $\to 2^{O(T(n))}$ 步 DTM（**指数级代价**）
单向无穷带 / 原地停留 S / 随机访存 / 文法 / μ-递归函数	左边界加特殊挡板字符；停留动作拆为一右一左两步；文法推导与带无界最小化算子 μ 的递归函数均可与 TM 互相模拟	识别能力均与标准 TM 等价

【易错警示】模拟非确定图灵机为什么必须用“广度优先（BFS）”而不能用“深度优先（DFS）”？
非确定图灵机的计算树上，可能第一条分支是无限死循环，而第二条分支只需 3 步就接受。如果按深度优先一条道走到黑，模拟机就会卡死在第一条无限分支里，永远发现不了旁边的接受分支！

▶ **丘奇–图灵论题（Church–Turing Thesis）**　直觉上“有效可计算”的任何过程，都可以由标准图灵机实现。注意：因为“直觉上的有效计算”不是形式化数学对象，所以它是一条被科学界广泛接受的**基本论题**，而不是能从七元组推导出来的数学定理。

▶ **图灵机编码 $\langle M\rangle$ 与通用图灵机 M_u**　每台图灵机的状态和转移规则都是有限的。按课件规则，可以把 $\delta(q_i, X_j) = (q_k, X_l, D_m)$ 编码为 $0^{i+1}10^j10^{k+1}10^l10^m$，再用 11 把所有规则拼成一串二进制编码 $\langle M\rangle$。

- **程序即数据**：既然图灵机 M 可以编码成普通字符串 $\langle M\rangle$，那么一台图灵机就能把另一台图灵机的编码（甚至它自己的编码！）当作输入来读取。
- **通用图灵机 M_u**：输入 $\langle M, w\rangle$，先检查 $\langle M\rangle$ 编码格式合法，然后在草稿带上维护 M 的纸带内容与当前状态，逐步查找 $\langle M\rangle$ 中的规则来模拟 M 在 w 上的运行。M_u 接受 $\langle M, w\rangle \iff M$ 接受 w。它正是现代存储程序通用计算机的理论原型。

3　可判定、可识别与枚举器

【课件对应】《08-图灵机和可计算》PDF 第 1–3 页（幻灯片 4–18）。

3.1　一张表分清“可判定（递归）”与“可识别（递归可枚举）”

在阅读教材和习题时，两组术语完全同义：**图灵可判定 ≡ 递归（Recursive）**；**图灵可识别 ≡ 递归可枚举（Recursively Enumerable, RE，或半可判定）**。

语言类别	当输入 $w \in L$ 时	当输入 $w \notin L$ 时	直观比喻
可判定（递归）	必在有限步停机并**接受**	必在有限步停机并**拒绝**	考官总能当场给出“过”或“挂”的明确裁决
可识别（RE）	必在有限步停机并**接受**	停机拒绝，或**永远运行不停机**	只发录取通知书;没收到的人分不清是落榜还是还在审

3.2　核心等价定理：L 可判定 $\iff$ L 与 $\overline{L}$ 都可识别

定理：L 是图灵可判定的 $\iff$ L 与补语言 $\overline{L} = \Sigma^* \setminus L$ 都是图灵可识别的

- **正向（⇒）**：若 L 可判定，取其判定器 D（对一切输入必停）。把 D 的接受状态与拒绝状态对调，得到的机器 D' 就是 $\overline{L}$ 的判定器，自然也是 $\overline{L}$ 的识别器。
- **反向（⇐）**：设 M_1 识别 L，M_2 识别 $\overline{L}$。构造机器 D：对输入 w，**交替模拟** $M_1(w)$ 和 $M_2(w)$（第 t 轮各走第 t 步）：
  - 一旦 M_1 进入**接受**状态，D 立即停机并**接受**；
  - 一旦 M_2 进入**接受**状态，D 立即停机并**拒绝**。

因为任意字符串 w 要么属于 L，要么属于 $\overline{L}$，所以 M_1 和 M_2 中**必有一台**会在有限步内进入接受状态！因此 D 绝不会死循环，且回答完全正确。

【易错警示】反向证明里的两个高频扣分点　1. **必须并行/交替模拟，绝不能串行先跑完 M_1 再跑 M_2**：因为当 $w \notin L$ 时，$M_1(w)$ 可能死循环，串行的话永远轮不到 M_2 运行。
2. **触发条件必须写“其中一台进入接受状态”，不能笼统写“其中一台停机”**：因为识别器在非成员输入上也可能“停机拒绝”。如果 $w \in \overline{L}$ 且 $M_1(w)$ 恰好停机拒绝，你必须等确认它是拒绝（或等 M_2 接受）后对 L 回答“否”，而不能一见停机就不分青红皂白地接受。

3.3　封闭性汇总与“串行 vs. 交错”证明模板

语言类	并 $A \cup B$	交 $A \cap B$	补 $\overline{A}$	连接 $A \cdot B$	星闭包 A^*
可判定（递归）	封闭	封闭	**封闭**	封闭	封闭
可识别（RE）	封闭	封闭	**不封闭**	封闭	封闭

为什么写封闭性证明时，可判定和可识别的写法不同？

- **可判定语言（判定器必停 → 放心串行枚举）**：因为每次调用子判定器都会在有限步返回明确结果，所以并、交可以先把 $D_1(w)$ 跑完再跑 $D_2(w)$；连接 $A \cdot B$ 只需把长 n 的输入 w 切成 $n+1$ 种 $w = uv$ 逐一测试；A^* 只需枚举有限种非空切分逐一测试。
- **可识别语言（识别器可能死循环 → 必须分时交错推进）**：
  - **并 $A \cup B$**：交替运行 $M_1(w)$ 与 $M_2(w)$，**任意一台接受**即接受（若一台先停机拒绝，继续单独跑另一台）。
  - **交 $A \cap B$**：可先运行 $M_1(w)$，等它接受后再运行 $M_2(w)$（或交替运行并用标记记录两台均已接受才接受）。
  - **连接 $A \cdot B$ 与闭包 A^***：把有限种切分方案全部开启为候选分支，每轮给每个候选切分方案各推进 1 步，只要有一种切分方案的所有分段都进入了接受状态，整体就接受。

3.4　枚举图灵机（Enumerator）：换一种角度看可识别与可判定

枚举器 E 是一台不读外部输入、从空工作带开始运行，并在输出带上源源不断打印 `#x#` 的图灵机，它打印出的所有字符串集合记为 $L(E)$。

1. **定理：L 图灵可识别 $\iff$ 存在枚举器 E 枚举 L（允许乱序、重复打印）**
   - **已知枚举器 E 造识别器 M**：输入 w，启动 E，每当 E 打印出一个串就和 w 比对；若相等则接受，否则继续等 E 打印下一个。
   - **已知识别器 M 造枚举器 E（燕尾法 / Dovetailing）**：把 Σ^* 中所有字符串按固定顺序排成 $w_1, w_2, w_3, \ldots$。不能先把 $M(w_1)$ 跑到底（万一 $M(w_1)$ 死循环就永远打不出 w_2 了）！正确做法是分阶段（$t = 1, 2, \ldots$）：在**第 t 阶段**，对前 t 个字符串 $w_1, w_2, \ldots, w_t$，分别模拟 $M(w_i)$ 的前 t 步，把其中进入接受状态的字符串打印出来。只要 $w_k \in L$，假设 $M(w_k)$ 在第 s 步接受，那么到了第 $t = \max(k, s)$ 阶段，w_k 一定会被打印出来！如果还不希望重复打印，只需在打印前先扫描输出带看是否已打印过即可。
2. **定理：L 图灵可判定 $\iff$ 存在枚举器按标准字典序（长度优先字典序 shortlex）递增打印 L**

- 若 L 是有穷集，直接硬编码有限表即可判定；
- 若 L 是无穷集，输入 w，运行递增枚举器：若在输出中看到了 w 就接受；若看到了一个**排在 w 之后的串**，说明 w 已经被跳过、永远不会再出现，立即停机拒绝！

3.5　基数论证：为什么世界上绝大多数语言都无法被图灵机识别？

无需构造具体问题，仅凭集合论就能看清计算能力的极限：

- **图灵机只有可数无穷多台**（$\aleph_0$）：每台图灵机都能编码成有限长的 01 字符串，而 Σ^* 中所有有限长字符串可以按长度一个个列出来，与自然数集 $\mathbb{N}$ 一一对应。
- **语言却有不可数无穷多个**（$2^{\aleph_0} = \aleph_1$）：每个语言都是 Σ^* 的一个子集，全体语言构成的幂集 $\mathcal{P}(\Sigma^*)$ 与实数集等势，不可数。

结论："问题（语言）"的数量远远多于"程序（图灵机）"的数量，因此必定存在大量连图灵识别器都没有的语言。下面第 4 节就把这些具体的"不可计算问题"抓出来。

4　不可判定性：对角化、映射归约与 Rice 定理

【课件对应】《08-图灵机和可计算》PDF 第 4–10 页（幻灯片 21–58）。

4.1　三大锚点问题：L_d、A_{TM}（或 L_{ACC}）与 H_{TM}（或 L_{HALT}）

在可计算性理论中，所有不可判定/不可识别证明都源于以下三个基础问题（课件记作 $L_d, L_{\mathrm{ACC}}, L_{\mathrm{HALT}}$，本文同时兼容常见记号 $A_{\mathrm{TM}}, H_{\mathrm{TM}}$）：

问题记号		定义	可判定?	可识别?	补可识别?
对角化语言 L_d		$\{w_i \mid w_i \notin L(M_i)\}$（第 i 机拒第 i 串）	否	**否**	是
接受问题（L_{ACC}）	A_{TM}	$\{\langle M, w\rangle \mid M$ 是图灵机且M 接受$w\}$	**否**	**是**	**否**
停机问题（L_{HALT}）	H_{TM}	$\{\langle M, w\rangle \mid M$ 是图灵机且M 在w 上停机$\}$	**否**	**是**	**否**

必背证明 1：为什么接受问题 $A_{\mathrm{TM}} = \{\langle M, w\rangle \mid M$ 接受$w\}$ 不可判定？（对角线法）

1. **为什么 A_{TM} 可识别？**直接运行通用图灵机 $M_u(\langle M, w\rangle)$：若 M 接受 w 则接受；若 M 拒绝或死循环则不接受。
2. **反证法证明 A_{TM} 不可判定**：假设存在一台总能停机的判定器 H 判定 A_{TM}：
$$H(\langle M, w\rangle) = \begin{cases} \text{接受}, & \text{若}M \text{ 接受}w, \\ \text{拒绝}, & \text{若}M \text{ 不接受}w\text{（含拒绝或不停机）}. \end{cases}$$
3. **构造"专门唱反调"的机器 D**：输入任意图灵机编码 $\langle M\rangle$，
   - 在 $\langle M, \langle M\rangle\rangle$ 上调用判定器 H（即问："机器 M 吃下自己的代码会不会接受?"）；
   - 若 H 回答"接受"，D 就故意**拒绝**；若 H 回答"拒绝"，D 就故意**接受**。
4. **把 D 自己的编码 $\langle D\rangle$ 喂给 D**：考察 $D(\langle D\rangle)$ 的结果：
$$D \text{ 接受}\langle D\rangle \iff H \text{ 判定}D \text{ 不接受}\langle D\rangle \iff D \text{ 不接受}\langle D\rangle,$$

产生逻辑悖论！因此假设不成立，判定器 H 根本不存在，A_{TM} 不可判定。

5. **推论：补语言 $\overline{A_{\mathrm{TM}}}$ 不可识别**。因为 A_{TM} 已经可识别，若 $\overline{A_{\mathrm{TM}}}$ 也都可识别，由 3.2 节核心定理就会推出 A_{TM} 可判定，矛盾！

▶ **必背证明 2：为什么停机问题 $H_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ 在} w \text{ 上停机}\}$ 不可判定？**　如果有人送给你一台能判定停机问题的机器 M_{HALT}，你就能用它轻松判定 A_{TM}：对输入 $\langle M, w\rangle$，**第一步**先用 M_{HALT} 检查 $M(w)$ 会不会停机；如果回答“不停机”，那么 M 绝不可能接受 w，直接回答“不接受”；如果回答“会停机”，**第二步**就放心在通用图灵机上实际模拟 $M(w)$ 直到它停下来（既然保证会停，模拟绝不会死循环），看它是接受还是拒绝即可。但我们已经证明 A_{TM} 不可判定，所以 M_{HALT} 也不可能存在！

4.2　映射归约 $A \le_m B$：方向怎么记才永远不会反？

映射归约（Mapping Reduction）的定义与传递法则

称语言 A 可映射归约到语言 B，记作 $A \le_m B$，如果存在一个**全可计算函数**（即总能停机输出结果的图灵机转换程序）$f : \Sigma^* \to \Sigma^*$，使得对一切字符串 x：

$$x \in A \iff f(x) \in B.$$

符号 $\le_m$ 的直观读法：“A 的难度 $\le$ B 的难度”（只要有办法解 B，就能先算 $f(x)$ 再解 A）：

- 若 $A \le_m B$ 且**右侧 B 容易**（可判定 / 可识别）$\Longrightarrow$ **左侧 A 也容易**（可判定 / 可识别）。
- 若 $A \le_m B$ 且**左侧 A 困难**（不可判定 / 不可识别）$\Longrightarrow$ **右侧 B 也困难**（不可判定 / 不可识别）。
- **补语言同步翻转**：$A \le_m B \iff \overline{A} \le_m \overline{B}$（同一个转换函数 f 自动满足 $x \notin A \iff f(x) \notin B$）。

【易错警示】归约方向口诀：“已知难问题 $\le_m$ 待证新问题”　想证明你手头的新问题 B 不可判定（或不可识别、NP 难），**永远要把已知难问题 A（如 A_{TM}）放在左边，把待证目标 B 放在右边，证明 $A \le_m B$！**
如果你证成了 $B \le_m A_{\mathrm{TM}}$，那就好比说“我可以调用原子弹来拍死苍蝇”，这只能说明拍苍蝇不比造原子弹更难，完全不能说明拍苍蝇本身很难。

4.3　构造辅助机器 N 的核心秘诀：“写代码”不等于“跑代码”！

用 $A_{\mathrm{TM}} \le_m B$ 证明某个图灵机性质 B 不可判定时，标准套路是：给定 A_{TM} 的输入 $\langle M, w\rangle$，转换函数 f 输出一台**新图灵机 N 的代码** $\langle N\rangle$。许多同学第一次学到这里都会疑惑：“既然 $M(w)$ 可能死循环，那 f 在构造 N 的时候怎么不会死循环?”

【直观理解】转换函数 f 只是“拼接代码的编译器”，它自己根本不运行 $M(w)$！　想象你在电脑上写一个 C 语言文件 `N.c`，里面写着：

```
void N(string x) { simulate(M, w); if (accepted) do_something(x); }
```

把固定的字符串 $\langle M\rangle$ 和 w 填空塞进这段代码模板、生成源码文件 $\langle N\rangle$，只需要几毫秒的文本替换，**必定停机**！至于 `simulate(M, w)` 会不会死循环，那是将来别人运行机器 N 时才会发生

的事，而我们甚至不需要真的去运行 N，只需要分析 N 所识别的语言 $L(N)$ 是什么！

【易错警示】写辅助机器 N 时绝对不能写的“非法伪代码” 在描述 $N(x)$ 的内部逻辑时，**绝对不能写**：“先模拟 $M(w)$，如果 M 永远不停机，则 N 转去执行某操作”。
为什么？因为当 $M(w)$ 死循环时，N 在模拟 $M(w)$ 的那一行就已经永远卡住了，根本不可能跳出循环去执行你的 `else` 分支！
正确的规范写法永远是：“模拟 $M(w)$；若模拟进入接受状态，则继续……；若模拟进入拒绝状态，则拒绝（若 $M(w)$ 死循环，则模拟自然永远运行下去，等价于 N 不接受 x）”。

4.4　四个经典不可判定/不可识别问题的归约表

利用上述“在 N 内部嵌入 $M(w)$”的模板，课件中的四个典型定理可以一览无余：

目标语言	给定 $\langle M, w\rangle$，辅助机器 $N(x)$ 怎么写？	当 M 接受 w 时 $L(N) =$	当 M 不接受 w 时 $L(N) =$
$L_{\mathrm{ACC01}} = \{\langle M\rangle \mid 01 \in L(M)\}$	输入 x：模拟 $M(w)$，若接受则接受 x（或仅当 $x = 01$ 时接受）	Σ^*（含 01，$\in L_{\mathrm{ACC01}}$）	$\varnothing$（不含 01，$\notin L_{\mathrm{ACC01}}$）
$L_e = \{\langle M\rangle \mid L(M) = \varnothing\}$	输入 x：模拟 $M(w)$，若接受则接受 x	$\Sigma^* \neq \varnothing$（$\notin L_e$，即 $\in \overline{L_e}$）	$\varnothing$（$\in L_e$）
$L_{\mathrm{REG}} = \{\langle M\rangle \mid L(M)$ 正则$\}$	输入 x：**若 $x \in \{0^n1^n\}$ 则直接接受**；否则模拟 $M(w)$，若接受则接受 x	Σ^*（正则，$\in L_{\mathrm{REG}}$）	$\{0^n1^n\}$（非正则，$\notin L_{\mathrm{REG}}$）
$L_{\mathrm{EQ}} = \{\langle M_1, M_2\rangle \mid L(M_1) = L(M_2)\}$	输出一对机器 $\langle M_1, N\rangle$，其中 N 同上（接受 Σ^* 或 $\varnothing$）：(1) 取 M_1 为空语言机器（$L(M_1) = \varnothing$）(2)取 M_1 为全语言机器（$L(M_1) = \Sigma^*$）	（1）$L(N) = \Sigma^* \neq L(M_1)$ （2）$L(N) = \Sigma^* = L(M_1)$	(1) $L(N) = \varnothing = L(M_1)$ (2) $L(N) = \varnothing \neq L(M_1)$

从最后一列可以读出两个极其深刻的结论：

- **空语言问题** L_e：上面的构造给出 $A_{\mathrm{TM}} \leq_m \overline{L_e} \iff \overline{A_{\mathrm{TM}}} \leq_m L_e$。因为 $\overline{A_{\mathrm{TM}}}$ 不可识别，所以 L_e **不仅不可判定，而且连图灵可识别都不是**！但它的补语言 $\overline{L_e} = \{\langle M\rangle \mid L(M) \neq \varnothing\}$ 是可识别的（只需用燕尾法交错模拟 M 在所有字符串上的运行，一旦发现某串被接受就接受）。
- **等价问题** L_{EQ}：构造（1）给出 $\overline{A_{\mathrm{TM}}} \leq_m L_{\mathrm{EQ}}$，构造（2）给出 $\overline{A_{\mathrm{TM}}} \leq_m \overline{L_{\mathrm{EQ}}}$。因此 L_{EQ} **和它的补语言** $\overline{L_{\mathrm{EQ}}}$ **全都不是图灵可识别的**（落在语言分类的最外层荒漠区）！

4.5　Rice 定理：一秒钟秒杀所有“语言性质”不可判定题

你一定发现了：上面证 $L_{\mathrm{ACC01}}, L_e, L_{\mathrm{REG}}$ 的套路简直一模一样！Rice 定理就是把这套模板直接打包成了通用定理：

莱斯定理（Rice's Theorem）及其两大适用前提
设 P 是关于图灵可识别语言的任意一个**非平凡性质**，则语言 $M_P = \{\langle M\rangle \mid L(M) \in P\}$ 必定**不可判定**。
使用 Rice 定理前，必须逐条核对以下**两个前提条件**（缺一不可）：

1. **语义性（只看语言 $L(M)$，不看机器代码实现）**：只要两台图灵机识别的语言相同（$L(M_1) =$

$L(M_2)$），无论它们内部有多少个状态、跑多少步，它们要么同时满足 P，要么同时不满足 P。

2. **非平凡性（不是全对，也不是全错）**：至少存在一个图灵可识别语言 $L_1 \in P$，且至少存在一个图灵可识别语言 $L_2 \notin P$。

▶ 考场真题演练：哪些能用 Rice 定理？哪些不能？（《08-图灵机和可计算》幻灯片 47–48）

待判断的图灵机集合	能否用 Rice?	原因与真实可判定性
$\{\langle M\rangle \mid M$ 至少接受37 个不同字符串$\}$	**能（✓）**	即 $\lvert L(M)\rvert \geq 37$，只依赖 $L(M)$ 且非平凡 $\Longrightarrow$ 不可判定
$\{\langle M\rangle \mid M$ 至少有37 个状态$\}$	**不能（×）**	关乎机器硬件规模而非 $L(M)$；直接数编码状态数即可，**可判定**
$\{\langle M\rangle \mid M$ 在输入01 上至多运行37 步$\}$	**不能（×）**	关乎运行步数而非 $L(M)$；模拟 37 步即可，**可判定**
$\{\langle M\rangle \mid M$ 在偶数步内接受01$\}$	**不能（×）**	依赖步数奇偶性，不满足条件 1；但本身**不可判定**（可由 L_{ACC01} 归约）
$\{\langle M\rangle \mid L(M)$ 可被某台偶数个状态的 TM 识别$\}$	**不能（×）**	任何可识别语言都能通过加一个无用状态变成偶数状态机器，故**所有 RE 语言都满足**（平凡性质！），恒为真，**可判定**
$\{\langle M\rangle \mid L(M)$ 可被某台 $\leq$ 37 状态且 $\leq$ 37 符号的 TM 识别$\}$	**能（✓）**	条件写的是 “$L(M)$ 属于某个有限语言族”，只依赖 $L(M)$ 且非平凡 $\Longrightarrow$ 不可判定
$\{\langle M\rangle \mid L(M)$ 可被某台 $\geq$ 37 状态且 $\geq$ 37 符号的 TM 识别$\}$	**不能（×）**	任何 TM 都可塞入无用状态和符号凑够 37 个，故所有 RE 语言都满足（平凡性质），**可判定**

▶ Rice 定理自身的证明骨架（万一考题不准直接引用 Rice，照此手写即可）　不妨设空语言 $\varnothing \notin P$（若 $\varnothing \in P$，先证补性质 $\overline{P}$ 不可判定即可）。因为 P 非平凡，必存在某个可识别语言 $L(M_1) \in P$。对任意 A_{TM} 的输入 $\langle M, w\rangle$，构造辅助机器 $M'(y)$：

> “输入 y：第一步先模拟 $M(w)$；若 $M(w)$ 接受，第二步再在输入 y 上模拟 $M_1(y)$，若 M_1 接受 y 则 M' 接受 y。”

- 若 $\langle M, w\rangle \in A_{\mathrm{TM}}$（$M$ 接受 w），则第一步必通过，$M'(y)$ 完全等价于 $M_1(y)$，故 $L(M') = L(M_1) \in P \Longrightarrow \langle M'\rangle \in M_P$；
- 若 $\langle M, w\rangle \notin A_{\mathrm{TM}}$（$M$ 不接受 w），则 M' 永远走不到第二步，故 $L(M') = \varnothing \notin P \Longrightarrow \langle M'\rangle \notin M_P$。

这给出了 $A_{\mathrm{TM}} \leq_m M_P$，故 M_P 不可判定。

4.6　图灵归约 $\leq_T$、波斯特对应问题（PCP）与王氏贴砖

1. **图灵归约 $A \leq_T B$ vs. 映射归约 $A \leq_m B$**：
   - **神谕图灵机（Oracle TM）M^B**：给图灵机外挂一个免费的 “黑盒问答机（神谕）”，每次把任意字符串 z 扔进黑盒，黑盒只需一步就能回答 $z \in B$ 是真是假。若存在总能停机的 M^B 判定 A，则称 $A \leq_T B$。
   - **为什么 $\leq_T$ 比 $\leq_m$ 更宽泛？** $A \leq_m B$ 只允许把输入 x 转换成**唯一一个**询问 $f(x)$，并且必须**原封不动照抄**黑盒的回答（不能把 Yes 改成 No！）。而 $A \leq_T B$ 允许查询多次，还允许把

黑盒的回答**取反**。因此：

$$A \le_m B \implies A \le_T B \quad \text{（反之不成立）}.$$

最典型的反例：$A_{\mathrm{TM}} \le_T \overline{A_{\mathrm{TM}}}$ 且 $\overline{A_{\mathrm{TM}}} \le_T A_{\mathrm{TM}}$（查一次黑盒再把答案取反即可），但 $A_{\mathrm{TM}} \not\le_m \overline{A_{\mathrm{TM}}}$（因为若 $A_{\mathrm{TM}} \le_m \overline{A_{\mathrm{TM}}}$，由于 A_{TM} 可识别，就会推出 $\overline{A_{\mathrm{TM}}}$ 可识别，矛盾!）。

2. **波斯特对应问题（PCP）与王氏贴砖（Domino Problem）：跳出图灵机外壳的不可判定问题**

- **PCP 定义**：给定 k 张上下写着字符串的骨牌 $(u_1, v_1), \ldots, (u_k, v_k)$，问是否存在一个**非空有限下标序列** $i_1, i_2, \ldots, i_m$（允许重复选同一张牌），使得上排拼起来的字符串等于下排拼起来的字符串：$u_{i_1} u_{i_2} \cdots u_{i_m} = v_{i_1} v_{i_2} \cdots v_{i_m}$。
- **证明链** $A_{\mathrm{TM}} \le_m$ MPCP $\le_m$ PCP：先定义强制要求第一张牌必须选 (u_1, v_1) 的修改版 MPCP。把图灵机 $M(w)$ 的初始配置 `#q0w#` 放进第一张牌的下排，利用转移规则设计骨牌，让上排总是落后下排一个 ID 配置，迫使上下排要想最终追平，就必须在骨牌上完整重现 $M(w)$ 从初始配置到接受状态 q_f 的计算历史（课件 PDF 第 9 页幻灯片 53）；再用星号插入技巧抹平“必须选第一张牌”的限制，归约到一般 PCP。因此 **MPCP 与 PCP 均不可判定**（但因为可以按长度枚举所有下标序列核对，所以 **PCP 是图灵可识别的**）。
- **王氏贴砖问题**：给定有限种四边涂色的正方形瓷砖（不可旋转），问能否铺满整个二维无限网格 $\mathbb{Z}^2$ 且相邻边颜色一致。该问题同样**不可判定**。

5　从可计算性转向计算复杂性：P、NP 与 NP 完全

【课件对应】《12-计算复杂性》PDF 第 1–4 页（幻灯片 4–23）。

可计算性只关心“有没有总能停机的算法”，但一个要跑 2^n 步的算法在现实中根本跑不完。计算复杂性理论专门在**可判定问题内部**进一步追问：**需要多少时间和空间才能算出答案？**

5.1　时间与空间复杂性的度量规则（最坏输入 + 最坏分支）

设输入字符串 x 的长度为 $n = |x|$，考察始终停机的 k 带图灵机 M：

- **确定型时间/空间**：若对每个长为 n 的输入，M 至多移动 $T(n)$ 步，则称 M 是 $T(n)$ 时间图灵机，它判定的语言属于 $\mathrm{DTIME}(T(n))$；若设第 1 条带为**只读输入带**，其余 $k-1$ 条存储（工作）带每条至多扫描 $S(n)$ 个格子，则称 M 是 $S(n)$ 空间图灵机，语言属于 $\mathrm{DSPACE}(S(n))$。
- **为什么空间复杂性要把输入带设为“只读且不计入空间”？**如果连输入带也算进空间，那只要把输入读一遍就至少用掉 n 个格子，永远无法定义亚线性空间（如对数空间 $O(\log n)$）！把输入带只读化后，专门统计工作带格子数，就能研究只需几个指针（$O(\log n)$ 位）的算法。
- **非确定型时间/空间**：对非确定图灵机 M，要求在长为 n 的输入上，**每一条可能的计算分支**都至多走 $T(n)$ 步（或每条工作带至多用 $S(n)$ 格），相应复杂性类记为 $\mathrm{NTIME}(T(n))$ 和 $\mathrm{NSPACE}(S(n))$。

由此定义两个最重要的时间复杂性类：

$$\mathrm{P} = \bigcup_{k \ge 1} \mathrm{DTIME}(n^k) = \mathrm{DTIME}(\mathrm{poly}), \qquad \mathrm{NP} = \bigcup_{k \ge 1} \mathrm{NTIME}(n^k) = \mathrm{NTIME}(\mathrm{poly}).$$

【易错警示】千万别把 NP 翻译成“非多项式时间（Non-Polynomial）”！ NP 是 **Nondeterministic Polynomial time（非确定型多项式时间）**的缩写！因为确定图灵机本身就是每次只有唯一转移分支的非确定图灵机，所以显然有 $\mathrm{P} \subseteq \mathrm{NP}$（一切多项式时间可解的简单问题全都在 NP 里）。至于 $\mathrm{P} = \mathrm{NP}$ 还是 $\mathrm{P} \neq \mathrm{NP}$，是计算机科学头号未解之谜。

5.2　理解 NP 的最佳方式：多项式时间验证器（Verifier）

与其把 NP 想成“非确定图灵机猜分支”，不如用等价的 **“猜出证书 + 快速验算”** 定义：

定理（NP 的验证器定义）：$L \in \mathrm{NP} \iff$ 存在多项式时间可验证的短证书

语言 $L \in \mathrm{NP}$，当且仅当存在一个确定的多项式时间算法（验证器）V 和多项式 p，使得对任意输入 x：

$$x \in L \iff \exists c \in \Sigma^*, \quad |c| \le p(|x|) \text{ 且 } V(x, c) = 1.$$

这里的字符串 c 称为 $x \in L$ 的**证书（Certificate / Witness，即“参考答案”）**。

【直观理解】自己做题很难，但批改别人的答案很快 比如给你一个 1000 个顶点的图 G，让你从零找出大小 $k = 50$ 的团，直接穷举有 $\binom{1000}{50}$ 种可能，慢如登天；但如果有人直接递给你一张小纸条 c（上面写着 50 个顶点编号，长度显然是输入的多项式），你只需花 $O(k^2)$ 时间检查这 50 个点是不是两两有边相连，瞬间就能验出真伪！因此 CLIQUE $\in$ NP。

5.3　判定问题 vs. 优化问题，以及 NP 完全性（NPC）的标准四步写法

课件幻灯片 18 专门对比了同一问题的三种问法（以团问题为例）：

- **判定问题（Decision）**：给定 $\langle G, k \rangle$，问 G 中**是否存在**大小 $\ge k$ 的团？（回答 Yes/No，对应一个语言，可以讨论是否属于 NP、是否 **NP 完全**）。
- **优化问题（Optimization）**：给定 G，问最大团的顶点数是多少？（输出数值，通常说 **NP 难**）。
- **计算/搜索问题（Search）**：给定 G，请把那个最大的团找出来。

▶ 多项式时间归约 $A \le_p B$、NP 难与 NP 完全的定义

- **多项式时间归约 $A \le_p B$**：存在一个在**确定多项式时间内可计算**的函数 $f : \Sigma^* \to \Sigma^*$，使得对任意 w，都有 $w \in A \iff f(w) \in B$。
- **B 是 NP 难（NP-Hard）**：对一切 $A \in \mathrm{NP}$，都有 $A \le_p B$（即 B 至少和 NP 里所有问题一样难）。
- **B 是 NP 完全（NP-Complete, NPC）**：同时满足两条——（1）$B \in \mathrm{NP}$；（2）B 是 NP 难的。

考场满分模板：证明新问题 B 是 NP 完全问题的固定四步

只要考卷上让你证明“某某问题 B 是 NP 完全的”，请严格按以下四步分段书写，阅卷老师一眼就能给全分：

1. **第一步：证明 $B \in \mathrm{NP}$**。明确写出：（1）证书 c 是什么（长度为何不超过 $|x|$ 的多项式）；（2）验证算法 $V(x, c)$ 具体检查哪几条性质，为什么能在多项式时间内完成。
2. **第二步：选定已知 NPC 问题 A 并写出归约构造 f**。声明“下面证明已知 NP 完全问题

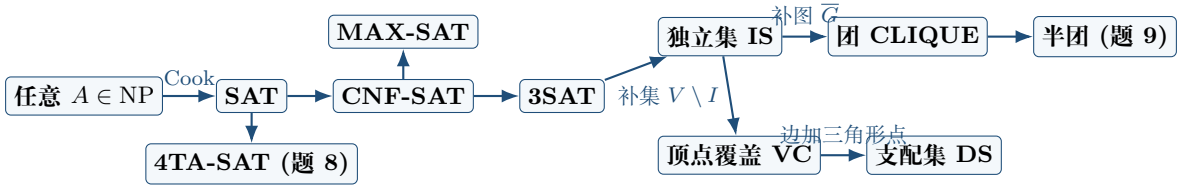
$A \le_p B$”（**切记方向是 $A \to B$，绝不能反！**），说清楚给定 A 的实例 x，如何一步步构造出 B 的实例 $f(x)$。

3. **第三步：双向证明正确性**（$x \in A \iff f(x) \in B$）。
   - **正向**（$\Rightarrow$）：设 $x \in A$（取 x 的一个解），说明如何据此在 $f(x)$ 中构造出满足 B 要求的解。
   - **反向**（$\Leftarrow$）：设 $f(x) \in B$（取 $f(x)$ 的任意一个解），说明为什么这个解不会“钻空子作弊”，必定能还原出原实例 x 的解。
4. **第四步：核算多项式时间代价**。用一句话指出构造出的 $f(x)$ 规模（顶点数、子句数等）是 $|x|$ 的多项式，且构造过程只需多项式时间。由归约传递性知 B 是 NP 完全问题。

6　Cook 定理与经典 NP 完全归约全景图

【课件对应】《12-计算复杂性》PDF 第 5–10 页、第 13–15 页（幻灯片 26–60, 76–88）。

整个 NP 完全性大厦的根基是 Cook 定理（证明了第一个 NPC 问题 SAT），随后通过多项式归约传递到逻辑、图论和数值划分问题。先把课件中的归约家族树装进脑子里：



6.1　Cook 定理（$\forall A \in \mathrm{NP},\ A \le_p \mathrm{SAT}$）：怎样用布尔公式模拟图灵机？

1. **SAT $\in$ NP**：证书是对公式中所有布尔变量的一组 0/1 赋值 τ，代入公式计算真值只需多项式时间。
2. **为什么任何 $A \in \mathrm{NP}$ 都能归约到 SAT？**设非确定图灵机 M 在多项式时间 $p(n)$ 内判定 A。给定输入 w（长 n），M 的任意一条计算分支最多跑 $p(n)$ 步，用到的纸带格子也绝不会超过 $p(n)+1$ 格。

   于是，我们可以把一条计算过程画成一张 $(p(n)+1) \times (p(n)+1)$ 的**二维时空网格表**：
   - 第 i 行（$0 \le i \le p(n)$）存放第 i 步的瞬时描述 ID_i；第 j 列存放第 j 个位置上的符号 $X_{i,j} \in Q \cup \Gamma$。
   - 为每个格子 (i,j) 和每个可能符号 $a \in Q \cup \Gamma$ 设立一个布尔变量 $y_{i,j,a}$：$y_{i,j,a}=1$ 表示“时刻 i 的第 j 格写着符号 a”。
3. **四组公式约束** $U \wedge S \wedge N \wedge F$：
   - **唯一与合法**（U）：每个格子 (i,j) 恰好有一个符号变量为 1，且每行恰好出现一个状态符号；
   - **初始配置**（S）：第 0 行固定为初始 ID $q_0 w B B \cdots B$；
   - **合法转移**（N）：图灵机一步只改动带头附近的局部格子，因此第 $i+1$ 行的每个 2×3 局部窗口是否合法，完全由第 i 行对应窗口根据转移函数 δ 决定，可写成局部布尔子句；
   - **最终接受**（F）：第 $p(n)$ 行中出现接受状态 q_f（规定进入 q_f 后原地空转补齐到 $p(n)$ 步）。

   整张表大小是 $O(p(n)^2)$，生成的布尔公式 $\varphi_{M,w,p(n)} = U \wedge S \wedge N \wedge F$ 长度也是 n 的多项式。M 存在接受 w 的计算分支 $\iff$ 存在一张合法的接受计算网格表 $\iff$ $\varphi_{M,w,p(n)}$ 可满足！

6.2　SAT $\leq_p$ CNF-SAT $\leq_p$ 3SAT：核心是“保可满足性（Equisatisfiable）”

把一般布尔公式化为合取范式（CNF，即“子句的与”），再化为每个子句恰好 3 个文字的 3CNF（作业题 7 必考！），如果硬用分配律 $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$ 展开，公式长度会指数爆炸。

【直观理解】为什么引入新变量能避免指数爆炸？ 归约只要求**“原公式 φ 可满足 $\iff$ 新公式 φ' 可满足”**（即存在原赋值的扩展赋值使新式为真），**并不要求两个公式在真值表上逻辑等价**！通过引入辅助变量拆分或补齐子句，公式长度只会线性增长。

▶ 第一步：SAT → CNF-SAT（课件 PDF 第 7 页幻灯片 39–40 的递归转换法）　先把否定号 $\neg$ 用德摩根律全部内推到单个变量上，然后对子表达式结构做归纳：

- 若 $E = E_1 \wedge E_2$，已分别转成子句集 $\{G_i\}$ 和 $\{H_j\}$（先把两边各自引入的同名辅助变量改名避免冲突），直接把两组子句拼起来：$\bigwedge_i G_i \wedge \bigwedge_j H_j$。
- 若 $E = E_1 \vee E_2$，已分别转成 $\bigwedge_{i=1}^{r} G_i$ 和 $\bigwedge_{j=1}^{s} H_j$，**引入一个全新辅助变量** y，构造 $r + s$ 个子句：

$$\bigwedge_{i=1}^{r} (G_i \vee y) \;\wedge\; \bigwedge_{j=1}^{s} (H_j \vee \neg y).$$

为什么对？不管 y 取 0 还是取 1，总有一半子句被 y 或 $\neg y$ 自动满足，另一半子句则强迫 E_1 或 E_2 必须为真！

▶ 第二步：CNF-SAT → 3SAT（把每个子句变成恰好 3 个文字，作业题 7 直接套用！）　对 CNF 中的每一个子句 C，按它的文字个数 m 分类改造（每个子句引入自己专属的新辅助变量）：

CNF 子句转 3CNF 的三条标准公式（必背）

1. $m = 1$（**单文字子句** x）：引入 2 个新变量 u, v，拆成 4 个子句（枚举 u, v 的全部 4 种真假组合）：
$$x \;\rightsquigarrow\; (x \vee u \vee v) \wedge (x \vee \neg u \vee v) \wedge (x \vee u \vee \neg v) \wedge (x \vee \neg u \vee \neg v).$$
（若 $x = 0$，无论 u, v 怎么取，4 个括弧里必有一个全为 0，从而迫使 x 必须为 1。）
2. $m = 2$（**二文字子句** $x \vee y$）：引入 1 个新变量 u，拆成 2 个子句：
$$(x \vee y) \;\rightsquigarrow\; (x \vee y \vee u) \wedge (x \vee y \vee \neg u).$$
3. $m = 3$（**三文字子句** $x \vee y \vee z$）：保持不变。
4. $m > 3$（**长子句** $x_1 \vee x_2 \vee \cdots \vee x_m$）：引入 $m - 3$ 个串联辅助变量 $y_1, y_2, \ldots, y_{m-3}$，拆成 $m - 2$ 个子句（像接力棒一样传递真值）：
$$(x_1 \vee x_2 \vee y_1) \wedge (x_3 \vee \neg y_1 \vee y_2) \wedge (x_4 \vee \neg y_2 \vee y_3) \wedge \cdots \wedge (x_{m-1} \vee x_m \vee \neg y_{m-3}).$$
（直观验证：如果原来的 $x_1, \ldots, x_m$ 全为 0，第 1 个子句逼迫 $y_1 = 1$，第 2 个子句逼迫 $y_2 = 1$ ……一路多米诺骨牌传到倒数第 2 个子句逼迫 $y_{m-3} = 1$，但最后一个子句只剩 $\neg y_{m-3} = 0$，当场矛盾！）

6.3　3SAT $\leq_p$ 独立集（IS）：怎样把逻辑公式翻译成一张图？

给定含 m 个子句的 3CNF 公式 $\varphi = e_1 \wedge e_2 \wedge \cdots \wedge e_m$，我们要构造图 $G = (V, E)$ 和整数 $k = m$，使得 φ 可满足 $\iff$ G 中存在大小至少为 m 的**独立集**（两两不相邻的顶点集）：

1. **建点（$3m$ 个顶点）**：每个子句 e_i 有 3 个文字位置，分别建顶点 $v_{i,1}, v_{i,2}, v_{i,3}$（哪怕同一个变量在不同子句出现多次，每次出现都单独建一个点）。
2. **连两类边（用“连边”代表“禁止同时选中”）**：
   - **A 类边（子句内部连成三角形）**：同一个子句的 3 个顶点两两连边。因为三角形里最多只能选 1 个不相邻的点，这就保证了：m 个三角形总共最多选 m 个点；若想凑够 $k = m$ 个点的独立集，**每个子句必须恰好被选中 1 个代表文字**！
   - **B 类边（矛盾文字之间连边）**：凡是一个顶点标着 x，另一个顶点标着 $\neg x$，就在它们之间连一条边。这就保证了：独立集绝不可能同时选中 x 和 $\neg x$，**选出来的 m 个代表文字在赋值上绝不自相矛盾**！
3. **双向验证一句话总结**：
   - φ 有满足赋值 τ $\Longrightarrow$ 从每个子句里挑一个被 τ 赋为 1 的文字顶点，它们既不在同一三角形内，也不可能是一正一反的矛盾文字，构成大小 m 的独立集。
   - G 有大小 m 的独立集 I $\Longrightarrow$ 鸽巢原理迫使 I 在每个子句三角形里各占 1 点，且无矛盾文字；把 I 中顶点对应的文字全部赋为 1（未涉及变量任取），即令全部 m 个子句为真！

6.4　独立集（IS）、团（CLIQUE）与顶点覆盖（VC）的三角铁关系

这三个图论问题之间的归约极其简短，但千万要分清**“对图取补图 $\overline{G}$”**和**“对顶点集取补集 $V \setminus I$”**的区别：

归约方向	构造映射 f	一句话等价原理
IS $\leq_p$ CLIQUE	$\langle G, k\rangle \mapsto \langle \overline{G}, k\rangle$ （**图取补图，阈值 k 不变**）	I 在原图 G 中两两无边 $\iff$ I 在补图 $\overline{G}$ 中两两有边（即形成团）
IS $\leq_p$ VC	$\langle G, k\rangle \mapsto \langle G, \lvert V\rvert - k\rangle$ （**图不变，阈值变为 $n-k$**）	I 中任意两点间无边 $\iff$ 图中每条边至少有一个端点落在 $V \setminus I$ 中（即 $V \setminus I$ 是顶点覆盖）

6.5　顶点覆盖（VC）$\leq_p$ 支配集（DS）：在每条边上架一个“三角形哨兵”

- **区别 VC 与 DS**：顶点覆盖要求**用点盖住所有边**（每条边至少一端被选）；支配集要求**用点管住所有点**（每个未选点至少与一个被选点相邻）。
- **为什么要给每条边加一个新顶点？**在原图 $G = (V, E)$ 中，对每条边 $e_i = (u, v) \in E$，新增一个辅助顶点 w_i，并连边 (u, w_i) 和 (w_i, v)（相当于在每条原边上搭了一个小三角形），得到新图 G'，阈值 k 不变（注：原图中的孤立点不连任何边，对 VC 无贡献，但在 DS 里却必须白白消耗名额去支配它；课件证明默认无孤立点/连通图，严谨起见可先删去原图孤立点）。
- **双向验证**：
  - VC $\Rightarrow$ DS：若 $I \subseteq V$ 是 G 的顶点覆盖，则每条边 (u, v) 至少有一端在 I 中，因此不仅原图所有非孤立点都与 I 相邻，所有新点 w_i 也都与 I 相邻，I 自然是 G' 的支配集。
  - DS $\Rightarrow$ VC：若 I' 是 G' 的大小 $\leq k$ 的支配集，万一 I' 里混进了某个新点 w_i，由于 w_i 只

跟 u,v 相邻，**把 w_i 替换成 u（或 v）**，支配范围只会变大不会变小，且集合大小 $\le k$ 不增！把所有新点都替换成原顶点后得到的 $I \subseteq V$ 仍要支配每个 w_i，而能支配 w_i 的原顶点只有 u 和 v，这就迫使每条边 (u,v) 至少有一端在 I 中，故 I 是原图的顶点覆盖！

6.6　哈密顿环（HAM-CYCLE）$\le_p$ 旅行商问题（TSP）：经典的 1/2 边权赋权法

给定无向图 $G=(V,E)$（设 $|V|=n\ge 3$），构造完全图 G'，边权函数 c 定义为：

$$c(u,v)=\begin{cases}1, & \text{若}(u,v)\in E\text{(原图有的边)},\\ 2, & \text{若}(u,v)\notin E\text{(原图没有的虚构边)},\end{cases}\qquad \text{令总预算阈值}k=n.$$

因为巡回圈必须恰好走过 n 条边，且每条边权至少为 1，所以**总权重大于等于 n，且总权重 $\le n$（即等于 n）当且仅当所走的 n 条边每一条的权重都严格等于 1**（即全部是原图 G 的真实边）！

6.7　课件其余 NP 完全问题速览表

问题名称	判定定义与核心复习要点
MAX-SAT / MAX-2SAT	给定子句集 F 和整数 k，问是否存在赋值满足**至少 k 个子句**。令 $k=\|F\|$ 即知 CNF-SAT 是 MAX-SAT 的特例。注意：普通 2SAT（要求全部满足）属于 P，但 MAX-2SAT 是 **NP 完全**的！（课件幻灯片 79 误贴了 MAX-SAT 的页，了解结论即可。）
NAE-SAT / NAE-3SAT	Not-All-Equal SAT：要求每个子句里**既有真文字又有假文字**（不能全为真，也不能全为假），均为 NP 完全。
边支配集 (EDS)	选至多 k 条边 E'，使每条未选边都与 E' 中某条边共享端点；NP 完全。
三维匹配 (3DM)	互不相交的 X,Y,Z 各有 n 个元素，$T\subseteq X\times Y\times Z$，问能否选出**大小恰好为 n** 的三元组子集 $M\subseteq T$ 使各三元组互不相交（对应二分图完美匹配的三维推广，2DM $\in$ P，但 3DM 是 NPC）。
子集和 (SUBSET-SUM)	给定正整数多重集 S 和目标 t，问是否存在子集和恰好为 t（NPC）。
划分 (PARTITION)	给定正整数多重集 S，问能否分成元素和相等的两半 S_1,S_2（即 $t=\frac{1}{2}\sum S$ 的子集和，NPC）。
多处理器调度 (MPS)	给定任务耗时多重集 S、处理器数 m 和截止时间 D，能否把 S 划分为 m 组使每组和 $\le D$（取 $m=2, D=\frac{1}{2}\sum S$ 即包含 PARTITION，NPC）。
循环有序三元组	给定集合 A 和三元组集合 $\mathcal{C}$，能否将 A 排在圆周（或 $1\sim\|A\|$）上，使每个 $(a,b,c)\in\mathcal{C}$ 满足 $f(a)<f(b)<f(c)$ 的三种循环移位之一（NPC）。

7　空间复杂性、博弈问题与对数空间类

【课件对应】《12-计算复杂性》PDF 第 10–12 页（幻灯片 61–75）。

7.1　复杂性类大一统层级与 Savitch 定理

把对数空间、多项式时间、多项式空间与指数时间排成一条链（课件中简写 PS = PSPACE，NPS = NPSPACE）：

$$\mathrm{L}\subseteq\mathrm{NL}=\mathrm{coNL}\subseteq\mathrm{P}\subseteq\mathrm{NP}\subseteq\mathrm{PSPACE}=\mathrm{NPSPACE}\subseteq\mathrm{EXPTIME}.$$

- **为什么空间比时间“更耐用”？(空间可复用，时间不可倒流)**：用过的时间步数一去不复返，但工作带上的格子擦掉之后可以反复重写！例如遍历一棵深度为 n 的完全二叉树（有 2^n 个叶节点），时间需要 $\Theta(2^n)$，但每次递归只需在工作带上记录当前从根到叶的一条路径，空间只需 $O(n)$。
- **为什么** PSPACE $\subseteq$ EXPTIME？一台只用 $p(n)$ 个格子的图灵机，纸带内容、带头位置和状态的全部组合数（即不同 ID 的总数）至多只有 $c^{p(n)} = 2^{O(p(n))}$ 种。根据鸽巢原理，如果它跑了超过 $c^{p(n)}$ 步还没停，就必定重复进入了同一个 ID 而陷入死循环！因此只要停机，步数必不超过指数级。
- **Savitch 定理** (PSPACE = NPSPACE)：在时间复杂性里我们不知道 P 是否等于 NP，但在空间复杂性里，非确定性最多只省平方级空间：NSPACE$(S(n)) \subseteq$ DSPACE$(S(n)^2)$。直观做法是：要判断配置 C_1 能否在 2^t 步内到达 C_2，只需枚举中间配置 C_{mid}，先递归检查 $C_1 \xrightarrow{2^{t-1}} C_{\text{mid}}$，再**擦掉刚才的工作空间**复用同一块区域检查 $C_{\text{mid}} \xrightarrow{2^{t-1}} C_2$。递归深度 $t = O(S(n))$，每层存一个配置用 $O(S(n))$，总空间仅 $O(S(n)^2)$！

7.2　PSPACE 完全问题：带量词布尔公式（QBF）与双人博弈

1. **QBF (Quantified Boolean Formula)**：SAT 只允许在最外层隐含存在量词 $\exists x_1 \exists x_2 \cdots$，而无自由变量的闭合 QBF 允许存在量词 $\exists$ 与全称量词 $\forall$ 交替出现，如：

$$\varphi = \exists x_1 \, \forall x_2 \, \exists x_3 \, \big[(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3)\big].$$

   注意量词顺序不可随便交换：$\forall x \, \exists y \, (x \leftrightarrow y)$ 为真（后手 y 可以照抄先手 x 的值），而 $\exists y \, \forall x \, (x \leftrightarrow y)$ 为假（先手固定 y 后，后手取 $x = \neg y$ 即令其为假）。判定闭合 QBF 是否为真是 **PSPACE 完全**问题。

2. **为什么 QBF 天然对应“双人轮流博弈”？**
   - **公式博弈 (Formula Game, FG)**：把 $\exists$ 变量交给玩家 E（想让公式最终为 1），把 $\forall$ 变量交给玩家 A（想让公式最终为 0），按量词从左到右的顺序轮流给变量赋 0 或 1。显然：**玩家 E 有必胜策略 $\iff$ 该 QBF 公式真值为** 1！因此 FG 是 PSPACE 完全的。
   - **广义地理学博弈 (Generalized Geography, GG)**：给定有向图 G 和起点 start，两名玩家轮流沿着有向边移动到下一个**此前从未访问过的顶点**（不能走回头路！），谁先无路可走谁输。判断先手玩家 I 是否有必胜策略也是 **PSPACE 完全**的。

7.3　P 完全与对数空间类（L 与 NL）：为什么这里必须换用“对数空间归约 $\leq_L$”？

当我们要在 P 内部找“最难并行化的问题（P 完全）”，或者在 NL 内部找“NL 完全问题”时，**绝对不能再用多项式时间归约** $\leq_p$！

【直观理解】为什么定义 P 完全和 NL 完全时，必须把归约工具降级为“对数空间归约 $\leq_L$”？
如果允许归约函数 f 自己随便花多项式时间，那在证明 $B \leq_p A$（其中 $B \in$ P）时，转换函数 f 干脆自己在多项式时间里把 B 的答案直接算出来，是就输出 A 的一个固定是实例，否就输出固定否实例——这样 P 里几乎随便挑个问题都能当“完全问题”了，毫无意义！
只有把归约函数限制在比 P 更弱的**对数工作空间**（$\leq_L$）内，才能真正反映 P 或 NL 内部的核心难度。

- **P 完全代表问题**：上下文无关文法空语言问题 $L_{\text{CFE}} = \{\langle G \rangle \mid G$ 是 CFG 且$L(G) = \varnothing\}$。
- **NL 完全代表问题**：有向图可达性问题 PATH $= \{\langle G, s, t \rangle \mid G$ 中存在从s 到t 的路径$\}$。

- **为什么** PATH $\in$ NL **？**非确定图灵机从当前顶点 $u = s$ 出发，每次非确定地猜下一个邻接顶点 v 并覆盖更新当前顶点 $u \leftarrow v$，同时维护一个步数计数器（至多计到 $|V|$）。工作带上只需存“当前顶点编号”和“步数计数器”两个整数，每个整数用二进制写只需 $O(\log |V|)$ 位！
- **Immerman–Szelepcsényi 定理**：课件还指出 PATH $\in$ coNL，进而 NL = coNL（注意：非确定机的补语言不能靠简单对调接受和拒绝状态得到，这是一个深刻的非平凡定理）。

8　考场证明题“防扣分”十条军规

在动笔做后面两节的 14 道作业题之前，先把以下十条高频失分点过一遍：

1. **看清机器要求**：题目若写“可判定（递归）”，构造的机器必须对一切输入停机；若写“可识别（RE）”，必须防止某个子过程死循环卡死全局（要用交替/分时模拟）。
2. **归约方向绝不写反**：证 B 难，永远是 已知难问题$A \le$ 待证问题B（即输入是 A 的实例，输出是 B 的实例）。
3. **不可判定归约别执行死循环判断**：构造的新机器 $N(x)$ 可以先模拟 $M(w)$、等 $M(w)$ 接受后再干别的；但绝不能写“若 $M(w)$ 死循环则 N 如何如何”。
4. **用 Rice 定理必写两句话**：（1）该性质只与图灵机识别的语言 $L(M)$ 有关；（2）举出一个满足它的可识别语言（如 Σ^* 或题目指定语言）和一个不满足它的可识别语言（如 $\varnothing$），说明非平凡。
5. **证 NP 完全必写四步**：（1）证 $\in$ NP（证书 + 多项式时间验证）；（2）写归约构造 $A \le_p B$；（3）双向证 $\Longleftrightarrow$；（4）说明多项式时间与规模。
6. **3CNF 转换盯紧每子句文字数**：转换后的每个子句必须**恰好含 3 个文字**（1 文字子句补 2 个新变量成 4 子句，2 文字子句补 1 个新变量成 2 子句，长子句串联拆分），且不同子句引入的辅助变量不能重名混用。
7. **区分补图与补集**：IS $\to$ CLIQUE 是翻转边得到补图 $\overline{G}$，阈值 k 不变；IS $\to$ VC 是同一张图取顶点补集 $V \setminus I$，阈值变为 $n - k$。
8. **图灵机 ID 别写反状态位置**：uqv 中带头正在读 v 的第一个字符；左移时状态跨到左边字符的前面。
9. **图灵机纸带中间别写空白** B：用 X, Y, Z 做删除标记，把 B 留给真正的左右端边界，回退左端时才不会认错路。
10. **边界输入顺手验一下**：写完图灵机或归约构造后，拿空串 ε、长度 1 的串或 $k \le 0$ 检验一遍，确保不翻车。

9　图灵机设计习题：5 道题完整转移表与详解

【课件对应】《图灵机设计习题》PDF 第 1–3 页。约定：统一使用单带确定图灵机，初始状态为 s（题 3 为 s_0），接受状态为 q_a，拒绝状态为 q_r；**凡转移表中未显式列出的 (状态, 字符) 组合，一律默认写回原符号、右移一格并转入拒绝状态** q_r。以下 5 台图灵机的转移表均已通过 Python 脚本对数千个边界与一般字符串完成穷举验证。

9.1　题 1：设计单带确定图灵机识别 $\{0^n1^n2^n \mid n \geq 0\}$，并写出 001122 的 ID 序列

【直观理解】每轮按序划掉一个 0、一个 1、一个 2，最后检查有且仅有标记符　用 X, Y, Z 分别代表已被配对划掉的 $0, 1, 2$。每一轮从左端出发：

1. 状态 s（找 0）：跳过已划掉的 X，遇到第一个 0 就把它改成 X，转入状态 a 去找 1；如果跳过 X 后直接撞见 Y（说明 0 已经全部配对完了），就转入状态 c 做收尾检查；如果在最开头直接遇到空白 B（说明输入是空串 ε，$n = 0$），直接接受！
2. 状态 a（找 1）：向右跨过剩下的 0 和已划掉的 Y，找到第一个 1 改成 Y，转入状态 b 去找 2。
3. 状态 b（找 2）：向右跨过剩下的 1 和已划掉的 Z，找到第一个 2 改成 Z，转入回退状态 r。
4. 状态 r（回退左端）：一路向左跨过所有 $0, 1, 2, X, Y, Z$，直到撞见左端空白 B，右移一格回到 s 开启下一轮。
5. 状态 c, d（收尾检查）：当 0 用光后，后面只允许依次是若干个 Y（状态 c）和若干个 Z（状态 d），最后遇到右端空白 B 才接受；若途中有残留的 $0, 1, 2$ 或顺序错乱，均因无转移规则而自动拒绝。

当前状态	读到符号	写入、移动 → 下一状态	该步动作的人话解释
s	X	$X, R \to s$	跳过左端已配对的 X
s	0	$X, R \to a$	找到本轮第一个未配对的 0，标为 X，去右边找 1
s	Y	$Y, R \to c$	0 已全部配对完毕，转入收尾阶段检查尾部是否全为 Y, Z
s	B	$B, R \to q_a$	输入为空串 ε（即 $n = 0$），直接接受
a	$0, Y$	原符号 $, R \to a$	跨过中间未配对的 0 和已配对的 Y
a	1	$Y, R \to b$	找到本轮配对的 1，标为 Y，继续向右找 2
b	$1, Z$	原符号 $, R \to b$	跨过中间未配对的 1 和已配对的 Z
b	2	$Z, L \to r$	找到本轮配对的 2，标为 Z，调头向左回退
r	$0, 1, 2, X, Y, Z$	原符号 $, L \to r$	一路向左退回纸带最左端
r	B	$B, R \to s$	碰到左端空白边界 B，右移一格回到起点开启下一轮
c	Y	$Y, R \to c$	收尾检查：跳过已配对的 Y
c	Z	$Z, R \to d$	收尾检查：从 Y 区进入 Z 区
d	Z	$Z, R \to d$	收尾检查：跳过已配对的 Z
c, d	B	$B, R \to q_a$	直到右端空白 B 均无多余未配对字符，停机接受

正确性与停机说明：循环不变量为“每完成一轮回到 s 时，纸带上 X, Y, Z 的数量始终相等，且相对顺序未被破坏”。每轮严格减少 1 个未标记的 0，至多 $|w|$ 轮后必进入收尾或拒绝，故对一切输入必

停机。

▶ **输入 $w = 001122$ 的完整瞬时描述（ID）转移序列**　以下逐步瞬时描述（ID）省略纸带两端未访问的空白格 B；表中未显式列出的转移默认写回原符号、右移一格并进入拒绝状态 q_r。

步数	瞬时描述（ID）
0	s001122
1	Xa01122
2	X0a1122
3	X0Yb122
4	X0Y1b22
5	X0Yr1Z2
6	X0rY1Z2
7	Xr0Y1Z2
8	rX0Y1Z2
9	rBX0Y1Z2
10	sX0Y1Z2
11	Xs0Y1Z2
12	XXaY1Z2
13	XXYa1Z2
14	XXYYbZ2
15	XXYYZb2
16	XXYYrZZ
17	XXYrYZZ
18	XXrYYZZ
19	XrXYYZZ
20	rXXYYZZ
21	rBXXYYZZ
22	sXXYYZZ
23	XsXYYZZ
24	XXsYYZZ
25	XXYcYZZ
26	XXYYcZZ
27	XXYYZdZ
28	XXYYZZdB
29	XXYYZZBq_aB

9.2　题 2：识别 $\{w \in \{0,1\}^* \mid w$ 中0 和1 数目相同$\}$，并写出 001010 的 ID 序列

【直观理解】抓住最左边第一个未标记字符：是 0 就去右边找 1 抵消，是 1 就去右边找 0 抵消　本题不要求 0 在 1 前面（次序任意）。我们用 X 表示已抵消的 0，用 Y 表示已抵消的 1：

1. 状态 s 从左向右跳过已抵消的 X, Y，找第一个未抵消的字符：
   - 若先看到 0：把它改成 X，进入状态 f_1（向右寻找一个未抵消的 1）；
   - 若先看到 1：把它改成 Y，进入状态 f_0（向右寻找一个未抵消的 0）；
   - 若一路跳过 X, Y 直接看到了右端空白 B：说明所有字符都成功两两抵消，接受！
2. 状态 f_1 向右跳过 $0, X, Y$，找到第一个 1 改成 Y 后转入 r 回退左端（若撞见 B 也没找到 1，说明 0 比 1 多，直接拒绝）；状态 f_0 完全对称。

当前状态	读到符号	写入、移动 → 下一状态	该步动作的人话解释
s	X, Y	原符号 $, R \to s$	跳过左端已经配对抵消的字符
s	0	$X, R \to f_1$	抓到最左未配对字符为 0，标为 X，去右边找 1 配对
s	1	$Y, R \to f_0$	抓到最左未配对字符为 1，标为 Y，去右边找 0 配对
s	B	$B, R \to q_a$	整条纸带已全部成对抵消完毕，停机接受
f_1	$0, X, Y$	原符号 $, R \to f_1$	正在找 1：跳过沿途的 0 和已标记字符 X, Y
f_1	1	$Y, L \to r$	成功找到配对的 1，标为 Y，调头向左回退
f_0	$1, X, Y$	原符号 $, R \to f_0$	正在找 0：跳过沿途的 1 和已标记字符 X, Y
f_0	0	$X, L \to r$	成功找到配对的 0，标为 X，调头向左回退
r	$0, 1, X, Y$	原符号 $, L \to r$	一路向左退回纸带最左端
r	B	$B, R \to s$	碰到左端空白 B，右移一格回到 s 开启下一轮

正确性与停机说明：每轮恰好同时消去一个 0 和一个 1，未标记字符中“0 的个数减去 1 的个数”保持不变，且未标记字符总数每轮减少 2，必在有限步停机。注意：题目给定的测试串 $w = 001010$ 含有 **4 个** 0、**2 个** 1，数目不等，因此机器在第 23 步扫描到右端空白 B 找不到更多 1 时**正确停机拒绝（进入 q_r）**。

► **输入 $w = 001010$ 的完整瞬时描述（ID）转移序列**　以下逐步瞬时描述（ID）省略纸带两端未访问的空白格 B；表中未显式列出的转移默认写回原符号、右移一格并进入拒绝状态 q_r。

步数	瞬时描述（ID）
0	s001010
1	X$f_1$01010
2	X0$f_1$1010
3	Xr0Y010
4	rX0Y010
5	rBX0Y010
6	sX0Y010
7	Xs0Y010
8	XXf_1Y010
9	XXY$f_1$010
10	XXY0$f_1$10
11	XXYr0Y0
12	XXrY0Y0
13	XrXY0Y0
14	rXXY0Y0
15	rBXXY0Y0
16	sXXY0Y0
17	XsXY0Y0
18	XXsY0Y0
19	XXYs0Y0
20	XXYXf_1Y0
21	XXYXY$f_1$0
22	XXYXY0f_1B
23	XXYXY0Bq_rB

9.3　题 3：设计单带确定图灵机识别 $\{0^{2^n} \mid n \geq 0\}$

【直观理解】每轮向右扫描“隔一个划掉一个 0”，把 0 的个数不断除以 2　注意：$n \geq 0$ 时，$2^n \in \{1, 2, 4, 8, \ldots\}$，因此单个 0（$n = 0, 2^0 = 1$）应接受，空串 ε 应拒绝！

怎样在不写计数器的情况下判断长度是不是 2 的幂？只需每一轮从左到右扫描尚未划掉的 0（已划掉的标为 X）：**保留第 1 个 0，把第 2 个 0 改成 X，保留第 3 个 0，把第 4 个 0 改成 X……**

- 如果本轮开始时恰好只剩 **1 个** 0，且后面已经碰到右端空白 B，说明一路除以 2 最终得到了 1，**接受**！
- 如果本轮有 ≥ 2 个 0，且的总数是**偶数**，那么扫到右端空白 B 时刚好完成最后一次“成对划掉”（处于状态 s_2），此时剩下的 0 恰好减半，回退左端继续下一轮！
- 如果本轮有 ≥ 3 个 0，但总数是**奇数**，那么扫到右端空白 B 时会停在“刚保留了一个落单的 0、正等着划掉下一个 0”的状态 s_3，抓到奇数因子，立即**拒绝**！

当前状态	读到符号	写入、移动 → 下一状态	该步动作的人话解释
s_0	X	$X, R \to s_0$	本轮刚起步（已见 0 个活跃 0）：跳过已划去的 X
s_0	0	$0, R \to s_1$	看到本轮第 1 个活跃 0，**保留不划**，转入 s_1
s_1	X	$X, R \to s_1$	已见 1 个活跃 0：跳过沿途的 X
s_1	B	$B, R \to q_a$	整条纸带恰好只剩 1 个活跃 0，长度是 2 的幂，**接受**
s_1	0	$X, R \to s_2$	看到本轮第 2 个活跃 0，**划成 X**（当前已见偶数个），转入 s_2
s_2	X	$X, R \to s_2$	已见 ≥ 2 且偶数个活跃 0：跳过沿途的 X
s_2	0	$0, R \to s_3$	看到下一个奇数位活跃 0，**保留不划**，转入 s_3
s_2	B	$B, L \to r$	扫到右端时活跃 0 恰为偶数个（已成功减半），调头回退
s_3	X	$X, R \to s_3$	已见 ≥ 3 且奇数个活跃 0：跳过沿途的 X
s_3	0	$X, R \to s_2$	看到下一个偶数位活跃 0，**划成 X**，回到偶数状态 s_2
r	$0, X$	原符号 $, L \to r$	一路向左退回纸带最左端
r	B	$B, R \to s_0$	碰到左端空白 B，右移一格回到 s_0 开启下一轮折半

正确性与停机说明：若 s_0 遇 B（空串）或 s_3 遇 B（大于 1 的奇数），均因未列规则直接转入 q_r 拒绝。每轮折半使活跃 0 的个数严格从 $2k$ 变为 k，至多 $\lfloor \log_2 |w| \rfloor + 1$ 轮必停机。

9.4　题 4：设计单带确定图灵机识别 $\{w\#w \mid w \in \{0,1\}^*\}$

【直观理解】用状态记住左半边刚划掉的是 0 还是 1，跨过 # 去右半边核对同位置字符　用 X 标记左半边已核对的字符，用 Y 标记右半边已核对的字符。每一轮：

1. 状态 s 在左半边跳过 X，读到第一个未核对字符：
   - 若是 0：改成 X，转入状态 a_0（脑子里记住“刚才拿的是 0”），向右去找 #；
   - 若是 1：改成 X，转入状态 a_1（脑子里记住“刚才拿的是 1”），向右去找 #；
   - 若直接遇到 #：说明左半边已经全部核对完了，转入状态 c 检查 # 右边是不是也恰好全是 Y。
2. 状态 a_0（或 a_1）向右跨过左半边剩余的 0,1，跨过分界符 # 后切换为状态 b_0（或 b_1）。

3. 状态 b_0（或 b_1）跳过右半边已核对的 Y，检查右半边第一个未核对字符：必须也是 0（或 1），将其改成 Y 并转入 r 回退左端；若不匹配或遇到 $B/\#$，自动拒绝！

当前状态	读到符号	写入、移动 → 下一状态	该步动作的人话解释
s	X	$X, R \to s$	跳过左半边已核对的字符 X
s	0	$X, R \to a_0$	左半边当前位是 0，标为 X，带着记忆 0 向右找 $\#$
s	1	$X, R \to a_1$	左半边当前位是 1，标为 X，带着记忆 1 向右找 $\#$
s	$\#$	$\#, R \to c$	左半边已全部核对完毕，跨过 $\#$ 检查右半边是否无剩余
a_0	$0, 1$	原符号 $, R \to a_0$	带着记忆 0 跨过左半边剩余的 $0, 1$
a_0	$\#$	$\#, R \to b_0$	跨过分界符 $\#$，进入右半边准备核对 0
a_1	$0, 1$	原符号 $, R \to a_1$	带着记忆 1 跨过左半边剩余的 $0, 1$
a_1	$\#$	$\#, R \to b_1$	跨过分界符 $\#$，进入右半边准备核对 1
b_0	Y	$Y, R \to b_0$	在右半边跳过已核对的 Y
b_0	0	$Y, L \to r$	右半边对应位也是 0，匹配成功！标为 Y 并调头回退
b_1	Y	$Y, R \to b_1$	在右半边跳过已核对的 Y
b_1	1	$Y, L \to r$	右半边对应位也是 1，匹配成功！标为 Y 并调头回退
r	$0, 1, Y, \#, X$	原符号 $, L \to r$	一路向左退回纸带最左端
r	B	$B, R \to s$	碰到左端空白 B，右移一格回到 s 开启下一轮
c	Y	$Y, R \to c$	收尾检查：跳过右半边已核对的 Y
c	B	$B, R \to q_a$	右半边也刚好全部核对完毕且无多余字符，停机接受

正确性与停机说明：若输入不含 $\#$、含多个 $\#$、左右长度不等或某位字符不同，均会在相应状态找不到转移而立即拒绝；输入仅一个 $\#$（对应 $w = \varepsilon$）则由 $s \xrightarrow{\#} c \xrightarrow{B} q_a$ 正确接受。

9.5　题 5：设计单带确定图灵机识别 $\{ww \mid w \in \{0,1\}^*\}$

【易错警示】千万不能直接比对首尾字符！首尾比对识别的是回文 $\{ww^R\}$，不是复制串 $\{ww\}$！ 例如 $0101 = 01 \cdot 01 \in \{ww\}$，但它的首字符是 0、尾字符是 1，并不相等！因为中间没有分界符 $\#$，我们必须分**两个阶段**完成：（1）**第一阶段（双向夹逼找中点）**：每轮把左端第一个未分类字符标为 L_0/L_1，再跑到右端把最后一个未分类字符标为 R_0/R_1，直到把偶长串精确分成前半段 L 区和后半段 R 区（奇长串会在右端找不到配对而当场拒绝）；（2）**第二阶段（同向逐位比对）**：像题 4 一样，依次拿 L 区的第 i 个字符与 R 区的第 i 个字符比对！

当前状态	读到符号	写入、移动 → 下一状态	该步动作的人话解释
第一阶段：从左右两端向中间夹逼，把前半段标为 L_0, L_1，后半段标为 R_0, R_1			
s	L_0, L_1	原符号 $, R \to s$	跳过左端已标为左半区的字符
s	0	$L_0, R \to p$	将最左未分类的 0 标为 L_0，去右端标一个右半区字符
s	1	$L_1, R \to p$	将最左未分类的 1 标为 L_1，去右端标一个右半区字符
s	R_0, R_1	原符号 $, L \to u$	左右两区已在正中间会师！调头回左端进入第二阶段比对
s	B	$B, R \to q_a$	输入为空串 ε（即 $w = \varepsilon$），直接接受
p	$0, 1$	原符号 $, R \to p$	向右跨过中间尚未分类的 $0, 1$
p	R_0, R_1, B	原符号 $, L \to q$	撞见右半区边界，退回一格对准最右未分类字符
q	0	$R_0, L \to r_1$	将最右未分类的 0 标为 R_0，调头回左端（若遇 L_b 则奇长拒绝）
q	1	$R_1, L \to r_1$	将最右未分类的 1 标为 R_1，调头回左端
r_1	$0, 1, L_0, L_1$	原符号 $, L \to r_1$	第一阶段回退：一路向左退回最左端
r_1	B	$B, R \to s$	碰到左端空白 B，右移回到 s 继续夹逼
第二阶段：依次取左半区的 L_b（划为 X）与右半区的 R_b（划为 Y）逐位比对			
u	X, Y, L_0, L_1	原符号 $, L \to u$	一路向左退回纸带最左端
u	B	$B, R \to m$	碰到左端空白 B，右移一格进入比对起点 m
m	X	$X, R \to m$	跳过左半区已比对成功的 X
m	L_0	$X, R \to m_0$	左半区当前位是 L_0，标为 X，带着记忆 0 去右半区找 R_0
m	L_1	$X, R \to m_1$	左半区当前位是 L_1，标为 X，带着记忆 1 去右半区找 R_1
m	Y	$Y, R \to q_a$	左半区已全部变成 X（撞见右半区的 Y），全部比对成功，**接受**
m_0	L_0, L_1, Y	原符号 $, R \to m_0$	跨过左半区剩余字符和右半区已比对的 Y
m_0	R_0	$Y, L \to u$	右半区对应位也是 R_0，标为 Y，调头回左端继续下一位
m_1	L_0, L_1, Y	原符号 $, R \to m_1$	跨过左半区剩余字符和右半区已比对的 Y
m_1	R_1	$Y, L \to u$	右半区对应位也是 R_1，标为 Y，调头回左端继续下一位

正确性与停机说明：第一阶段每轮将未分类字符减少 2 个，若原长 $|w|$ 为奇数，最后一轮 s 把唯一剩余字符标为 L_b 后，p 右移再左移会使 q 读到 L_b，因无转移规则而立即拒绝；若 $|w| = 2k$ 为偶数，第一阶段结束后纸带化为 $L_{b_1} \cdots L_{b_k} R_{c_1} \cdots R_{c_k}$，第二阶段严格按序比对 $b_i = c_i$（$1 \le i \le k$），完全相等才接受。

10　可计算与复杂性理论习题：9 道题规范证明全解

【课件对应】《可计算和复杂性理论习题》PDF 第 1–2 页。以下证明均按考场满分步骤编写，可直接作为答题范本。

10.1　题 1：证明递归语言在并操作下封闭（若 L, L' 均递归，则 $L \cup L'$ 也递归）

证明：

1. 因为 L 和 L' 都是递归语言（即图灵可判定语言），根据定义，分别存在始终停机的确定图灵机（判定器）M_1 和 M_2，使得 $L(M_1) = L$，$L(M_2) = L'$。
2. 构造一台新的图灵机 M，对任意输入字符串 $w \in \Sigma^*$ 按如下步骤运行：
   - **第一步**：在输入 w 上模拟运行 $M_1(w)$。因为 M_1 是判定器，它必在有限步内停机。若 M_1 停机并**接受**，则 M 立即停机并**接受**；
   - **第二步**：若 M_1 停机并**拒绝**，则在输入 w 上继续模拟运行 $M_2(w)$。因为 M_2 也是判定器，它也必在有限步内停机。若 M_2 接受，则 M 停机**接受**；若 M_2 拒绝，则 M 停机**拒绝**。
3. **停机性与正确性验证**：由于 M_1, M_2 对所有输入都必在有限步停机，故 M 对所有输入也必在有限步停机；且 M 接受 $w \iff M_1$ 接受 w 或 M_2 接受 $w \iff w \in L$ 或 $w \in L' \iff w \in L \cup L'$。因此 M 判定 $L \cup L'$，即 $L \cup L'$ 是递归语言。 □

10.2　题 2：证明递归可枚举语言在并操作下封闭（若 L, L' 均递归可枚举，则 $L \cup L'$ 也递归可枚举）

【易错警示】为什么题 2 绝对不能照抄题 1 的“先跑完 M_1 再跑 M_2”？ 在题 2 中，M_1 和 M_2 只是**识别器**，当 $w \notin L$ 时，$M_1(w)$ 可能会**永远死循环**！如果先串行等待 $M_1(w)$ 跑完，那么对于满足 $w \notin L$ 但 $w \in L'$ 的输入，M 就会永远卡在第一步的死循环里，永远轮不到 $M_2(w)$ 去接受它！因此**必须采用分时交替（并行）模拟**。

证明：

1. 因为 L 和 L' 都是递归可枚举语言（即图灵可识别语言），分别存在图灵识别器 M_1 和 M_2，满足 $L(M_1) = L$，$L(M_2) = L'$。
2. 构造一台新的图灵机 M，对任意输入 $w \in \Sigma^*$，分轮次 $t = 1, 2, 3, \ldots$ **交替模拟** $M_1(w)$ 与 $M_2(w)$：
   - 在第 t 轮，若 $M_1(w)$ 尚未停机拒绝，则将 $M_1(w)$ 向前模拟推进 1 步；若 $M_2(w)$ 尚未停机拒绝，则将 $M_2(w)$ 向前模拟推进 1 步。
   - 在任意时刻，只要 M_1 或 M_2 中**有任何一台进入接受状态**，M 就立即停机并**接受** w。
   - 若其中一台进入拒绝状态停机，则继续单独模拟另一台；若两台均已停机拒绝，则 M 停机拒绝。
3. **正确性验证**：
   - 若 $w \in L \cup L'$，则 $w \in L$ 或 $w \in L'$，故 $M_1(w)$ 与 $M_2(w)$ 中至少有一台会在某个有限步数 k 进入接受状态；由于交替推进，M 至多在第 k 轮就会检测到该接受状态并停机接受。
   - 若 $w \notin L \cup L'$，则 $M_1(w)$ 与 $M_2(w)$ 都永远不会进入接受状态，故 M 也绝不会接受 w。

   综上，$L(M) = L \cup L'$，即 $L \cup L'$ 是递归可枚举语言。 □

10.3　题 3：证明集合 $S = \{\langle M \rangle \mid L(M) = \{ww^R \mid w \in \{0,1\}^*\}\}$ 是不可判定的

记偶数长度回文语言为 $K = \{ww^R \mid w \in \{0,1\}^*\}$。下面给出两种满分证法（考场上任选其一，或两种都写最保险）：

▶ 证法一：直接应用 Rice 定理（最快）

1. **验证只依赖识别语言（语义性质）**：集合 S 的定义条件为 $L(M) = K$，完全由图灵机 M 所识别的语言 $L(M)$ 决定，与 M 的具体状态数或转移细节无关。
2. **验证非平凡性**：
   - 一方面，K 显然是可判定（从而也是图灵可识别）语言：图灵机只需反复比对并消去首尾字符，若全部消空则接受，否则拒绝。因此存在识别 K 的图灵机 M_1，满足 $L(M_1) = K$，即 $\langle M_1\rangle \in S$ $(S \neq \varnothing)$。
   - 另一方面，取不接受任何输入的空语言图灵机 M_0，有 $L(M_0) = \varnothing \neq K$（因为空串 $\varepsilon \in K$），故 $\langle M_0\rangle \notin S$。
3. 由 Rice 定理可知，S 是不可判定的。 □

▶ 证法二：从接受问题 A_{TM} 构造映射归约 $A_{\mathrm{TM}} \leq_m S$（不依赖 Rice 定理）

1. 先固定一台判定语言 $K = \{ww^R \mid w \in \{0,1\}^*\}$ 的图灵机 M_K。
2. 对 A_{TM} 的任意输入实例 $\langle M, w\rangle$，构造一台新图灵机 N 的编码 $\langle N\rangle = f(\langle M, w\rangle)$，$N$ 在自己的输入 $x \in \{0,1\}^*$ 上按如下两步工作：
   - **第一步**：忽略 x，在内部模拟 M 在固定串 w 上的运行 $M(w)$；
   - **第二步**：若 $M(w)$ 停机并**接受**，则在输入 x 上模拟运行 $M_K(x)$，当 $x \in K$ 时接受，否则拒绝。（若 $M(w)$ 拒绝或死循环，则 N 也直接拒绝或死循环，总之不接受任何 x。）
3. **验证** $\langle M, w\rangle \in A_{\mathrm{TM}} \iff \langle N\rangle \in S$：
   - 若 $\langle M, w\rangle \in A_{\mathrm{TM}}$（即 M 接受 w），则第一步必通过，N 对输入 x 接受当且仅当 $x \in K$，故 $L(N) = K$，从而 $\langle N\rangle \in S$；
   - 若 $\langle M, w\rangle \notin A_{\mathrm{TM}}$（即 M 不接受 w），则 N 永远无法通过第一步，不接受任何字符串，故 $L(N) = \varnothing \neq K$（因为 $\varepsilon \in K$），从而 $\langle N\rangle \notin S$。
4. 写出机器代码 $\langle N\rangle$ 的转换函数 f 显然全可计算，故 $A_{\mathrm{TM}} \leq_m S$。因 A_{TM} 不可判定，故 S 不可判定。□

10.4　题 4：证明集合 $T = \{\langle M\rangle \mid M$ 在所有输入上均停机接受$\}$ 是不可判定的

审题关键：按照图灵机接受的定义，“M 接受 x”本身就意味着“M 在 x 上于有限步内停机并进入接受状态”。因此，“M 在所有输入 $x \in \Sigma^*$ 上均停机接受”**完全等价于** $L(M) = \Sigma^*$！

▶ 证法一：从 A_{TM} 构造映射归约 $A_{\mathrm{TM}} \leq_m T$

1. 对 A_{TM} 的任意输入实例 $\langle M, w\rangle$，构造一台新图灵机 N（令 $f(\langle M, w\rangle) = \langle N\rangle$），$N$ 对任意输入 $x \in \Sigma^*$ 按如下方式运行：

   > 忽略自身输入 x，直接在工作带上模拟 M 在 w 上的运行 $M(w)$；若 $M(w)$ 停机接受，则 N 停机接受 x。

2. **双向验证**：
   - 若 $\langle M, w\rangle \in A_{\mathrm{TM}}$，则 $M(w)$ 在有限步内停机接受，于是对任意输入 $x \in \Sigma^*$，$N(x)$ 都在有限步内停机接受（即 $L(N) = \Sigma^*$），故 $\langle N\rangle \in T$；
   - 若 $\langle M, w\rangle \notin A_{\mathrm{TM}}$，则 $M(w)$ 永不接受，于是 N 在任何输入 x 上都不会停机接受（$L(N) = \varnothing \neq \Sigma^*$），故 $\langle N\rangle \notin T$。
3. 因 f 全可计算，故 $A_{\mathrm{TM}} \leq_m T$。由于 A_{TM} 不可判定，故 T 不可判定。 □

▶ 证法二：应用 Rice 定理　由于“M 在所有输入上均停机接受”等价于 $L(M) = \Sigma^*$，该性质只依赖识别语言 $L(M)$；且全接受机器满足该性质（$L = \Sigma^*$），全拒绝机器不满足该性质（$L = \varnothing$），故为非平凡语言性质，由 Rice 定理直接得 T 不可判定。 □

【易错警示】顺手辨析：如果题目改成“M 在所有输入上均停机（无论接受还是拒绝）”，还能用 Rice 吗？ 不能直接用语言版 Rice 定理！因为“所有输入停机”不仅看接受什么，还看非成员上是停机拒绝还是死循环（两台同样识别 $\varnothing$ 的机器，一台直接停机拒绝，另一台死循环，停机性不同）。但无论题目有无疑问“接受”，上面的**证法一（让 N 模拟 $M(w)$，若 $M(w)$ 拒绝则故意令 N 进入死循环）**都通杀有效！

10.5　题 5：证明最大团（Clique）问题是 NP 完全问题

审题说明：在形式语言与 NP 完全性理论中，团问题的标准判定形式为 $\mathrm{CLIQUE} = \{\langle G, k\rangle \mid$ 无向图$G = (V, E)$ 中存在大小至少为k 的团$\}$（若指求最大团大小的优化问题，则称为 NP 难）。下面按标准四步证明 CLIQUE 是 NP 完全问题：

1. **第一步：证明** $\mathrm{CLIQUE} \in \mathrm{NP}$。
   - **证书** c：取 G 的一个顶点子集 $C \subseteq V$，证书长度显然为 $O(|V| \log |V|)$，是输入长度的多项式。
   - **多项式时间验证器** V：收到 $\langle G, k\rangle$ 和顶点子集 C 后，（1）检查 C 中互异顶点的个数是否满足 $|C| \geq k$；（2）枚举 C 中所有顶点对 $\{u, v\}$（共 $\binom{|C|}{2} = O(|V|^2)$ 对），逐一在边集 E 中检查是否有边 $(u, v) \in E$。若两条均满足则输出 1，否则输出 0。验证耗时 $O(|V|^2)$，为多项式时间，故 $\mathrm{CLIQUE} \in \mathrm{NP}$。
2. **第二步：构造归约** $\mathrm{IS} \leq_p \mathrm{CLIQUE}$。选取已知的 NP 完全问题——独立集问题 $\mathrm{IS} = \{\langle G, k\rangle \mid G$ 中存在大小至少为k 的独立集$\}$（课件已由 $\mathrm{3SAT} \leq_p \mathrm{IS}$ 证得）。对 IS 的任意输入实例 $\langle G, k\rangle$（其中 $G = (V, E)$），构造转换函数 $f(\langle G, k\rangle) = \langle \overline{G}, k\rangle$，其中补图 $\overline{G} = (V, \overline{E})$ 的顶点集仍为 V，边集定义为：
$$\overline{E} = \big\{\{u, v\} \mid u, v \in V,\ u \neq v,\ \text{且} \{u, v\} \notin E\big\}.$$
3. **第三步：双向证明** $\langle G, k\rangle \in \mathrm{IS} \iff \langle \overline{G}, k\rangle \in \mathrm{CLIQUE}$。对任意顶点子集 $S \subseteq V$（$|S| \geq k$）：
$$\begin{aligned} S \text{ 是} G \text{ 的独立集} &\iff \forall u, v \in S\ (u \neq v),\ \{u, v\} \notin E \\ &\iff \forall u, v \in S\ (u \neq v),\ \{u, v\} \in \overline{E} \iff S \text{ 是} \overline{G} \text{ 的团}. \end{aligned}$$
   因此 G 有大小 $\geq k$ 的独立集当且仅当 $\overline{G}$ 有大小 $\geq k$ 的团。
4. **第四步：多项式时间分析**。构造补图 $\overline{G}$ 只需遍历所有 $\binom{|V|}{2}$ 个顶点对并翻转边的存在性，耗时 $O(|V|^2)$，属于多项式时间。综上，CLIQUE 是 NP 完全问题。 □

10.6　题 6：证明若 A 是 NP 完全问题且有多项式时间算法，则 $\mathrm{P} = \mathrm{NP}$

证明：

1. **已知** $\mathrm{P} \subseteq \mathrm{NP}$：根据定义，任何能被确定图灵机在多项式时间内判定的语言，自然也能被非确定图灵机（只需不使用非确定分支）在多项式时间内判定，故 $\mathrm{P} \subseteq \mathrm{NP}$ 恒成立。下面只需证明 $\mathrm{NP} \subseteq \mathrm{P}$。
2. **任取** $B \in \mathrm{NP}$，**证明** $B \in \mathrm{P}$：
   - 因为 A 是 NP 完全问题，根据 NP 完全的定义，对 NP 中的任意语言 B，都存在一个多项式时间可计算的归约函数 $f : \Sigma^* \to \Sigma^*$，使得对一切输入 $x \in \Sigma^*$，都有 $x \in B \iff f(x) \in A$。设算法计算 $f(x)$ 的时间上界为 $O(|x|^a)$（其中 $a \geq 1$ 为常数），由于每步最多写一个字符，输出字符串 $y = f(x)$ 的长度必满足 $|y| \leq O(|x|^a)$。
   - 又已知 A 存在多项式时间判定算法 M_A，设 M_A 在长度为 m 的输入上运行时间上界为 $O(m^b)$（其中 $b \geq 1$ 为常数）。
   - 现在为语言 B 构造确定型判定算法 M_B：对输入 x（设 $n = |x|$），**第一步**先调用归约算法算出 $y = f(x)$；**第二步**在 y 上运行 $M_A(y)$，并直接输出 $M_A(y)$ 的判定结果。
3. **核算正确性与总耗时**：
   - 正确性：M_B 接受 $x \iff M_A$ 接受 $f(x) \iff f(x) \in A \iff x \in B$。

- 运行时间：第一步耗时 $O(n^a)$，且 $|y| \le O(n^a)$；第二步耗时 $O(|y|^b) = O((n^a)^b) = O(n^{ab})$。因此 M_B 的总运行时间为 $O(n^a + n^{ab})$，仍然是关于输入长度 n 的多项式！

4. 这说明任意 $B \in \mathrm{NP}$ 都属于 P，即 $\mathrm{NP} \subseteq \mathrm{P}$。结合 $\mathrm{P} \subseteq \mathrm{NP}$，立即得到 $\mathrm{P} = \mathrm{NP}$。 □

10.7　题 7：将下列两个布尔表达式转换为保持可满足性一致的 3CNF 形式

核心法则回顾：直接套用 6.2 节的四条标准公式——单文字子句补 2 个新变量拆成 4 个子句，二文字子句补 1 个新变量拆成 2 个子句，m 文字长子句（$m > 3$）引入 $m-3$ 个串联新变量拆成 $m-2$ 个子句！

▶ **第 (1) 题：** $\varphi_1 = (x \vee y) \wedge z \wedge (\neg x \vee \neg y \vee z \vee a \vee b \vee \neg c)$　原式已经是合取范式（CNF），包含 3 个子句，长度分别为 2、1、6。我们引入互不相同的全新辅助变量 u, v, w, t_1, t_2, t_3，将三个子句分别改造如下：

1. **第一子句 $(x \vee y)$（2 文字 → 引入 u 拆为 2 个子句）**：

$$(x \vee y \vee u) \ \wedge \ (x \vee y \vee \neg u).$$

2. **第二子句 z（1 文字 → 引入 v, w 拆为 4 个子句）**：

$$(z \vee v \vee w) \ \wedge \ (z \vee v \vee \neg w) \ \wedge \ (z \vee \neg v \vee w) \ \wedge \ (z \vee \neg v \vee \neg w).$$

3. **第三子句 $(\neg x \vee \neg y \vee z \vee a \vee b \vee \neg c)$（6 文字 → 引入 t_1, t_2, t_3 拆为 $6-2=4$ 个子句）**：

$$(\neg x \vee \neg y \vee t_1) \ \wedge \ (z \vee \neg t_1 \vee t_2) \ \wedge \ (a \vee \neg t_2 \vee t_3) \ \wedge \ (b \vee \neg c \vee \neg t_3).$$

最终答案：将上述 $2+4+4=10$ 个三文字子句用 $\wedge$ 连接起来即为所求 3CNF：

$$\begin{aligned}\sigma_1 = {} & (x \vee y \vee u) \wedge (x \vee y \vee \neg u) \\ & \wedge (z \vee v \vee w) \wedge (z \vee v \vee \neg w) \wedge (z \vee \neg v \vee w) \wedge (z \vee \neg v \vee \neg w) \\ & \wedge (\neg x \vee \neg y \vee t_1) \wedge (z \vee \neg t_1 \vee t_2) \wedge (a \vee \neg t_2 \vee t_3) \wedge (b \vee \neg c \vee \neg t_3).\end{aligned}$$

▶ **第 (2) 题：** $\varphi_2 = (x \wedge y) \vee (\neg x \wedge z)$　原式不是 CNF（外层是“或”）。有两种完全正确的标准做法，考场上推荐把两种都写上：

- **方法 A（课件幻灯片 40 的引入辅助变量法，不展开分配律）**：先看成 $E_1 \vee E_2$，其中 $E_1 = x \wedge y$，$E_2 = \neg x \wedge z$。按照课件规则引入新变量 p，将 E_1 的每个子句析取 p，将 E_2 的每个子句析取 $\neg p$，化为等价可满足的 2CNF：

$$(x \vee p) \ \wedge \ (y \vee p) \ \wedge \ (\neg x \vee \neg p) \ \wedge \ (z \vee \neg p).$$

  再为这 4 个二文字子句分别引入全新辅助变量 u_1, u_2, u_3, u_4 补齐为 3CNF（共 8 个子句）：

$$\begin{aligned}\sigma_2 = {} & (x \vee p \vee u_1) \wedge (x \vee p \vee \neg u_1) \ \wedge \ (y \vee p \vee u_2) \wedge (y \vee p \vee \neg u_2) \\ & \wedge (\neg x \vee \neg p \vee u_3) \wedge (\neg x \vee \neg p \vee \neg u_3) \ \wedge \ (z \vee \neg p \vee u_4) \wedge (z \vee \neg p \vee \neg u_4).\end{aligned}$$

- **方法 B（直接用分配律化为 CNF 后再补齐）**：用分配律展开 $(x \wedge y) \vee (\neg x \wedge z) \equiv (x \vee \neg x) \wedge (x \vee z) \wedge (y \vee \neg x) \wedge (y \vee z)$，消去恒真式 $(x \vee \neg x)$ 后得到 3 个二文字子句 $(x \vee z) \wedge (\neg x \vee y) \wedge (y \vee z)$；再引入辅助变量 u_1, u_2, u_3 补齐为 6 个子句：

$$(x \vee z \vee u_1) \wedge (x \vee z \vee \neg u_1) \wedge (\neg x \vee y \vee u_2) \wedge (\neg x \vee y \vee \neg u_2) \wedge (y \vee z \vee u_3) \wedge (y \vee z \vee \neg u_3).$$

10.8　题 8：证明 4TA-SAT（至少有 4 个不同满足赋值）是 NP 完全问题

题目定义：4TA-SAT $= \{\langle \varphi \rangle \mid$ 布尔表达式φ 至少存在4 个互不相同的可满足赋值$\}$。

1. **第一步：证明 4TA-SAT $\in$ NP。**

- **证书** c：针对 φ 中变量集 $\mathrm{var}(\varphi)$ 的 4 组赋值 $(\tau_1,\tau_2,\tau_3,\tau_4)$。设变量数为 $k \le |\varphi|$，则证书总长度为 $4k = O(|\varphi|)$，是输入长度的多项式。
- **多项式时间验证器** V：（1）两两比对这 4 组赋值（共 $\binom{4}{2}=6$ 对），确认 $\tau_1,\tau_2,\tau_3,\tau_4$ 互不相同；（2）将 $\tau_1,\tau_2,\tau_3,\tau_4$ 分别代入布尔表达式 φ 求值，确认 $\tau_i(\varphi)=1$（$i=1,2,3,4$）。这两步均可在 $O(|\varphi|)$ 多项式时间内完成，故 4TA-SAT $\in$ NP。

2. **第二步：构造多项式时间归约** SAT $\le_p$ **4TA-SAT**。已知 SAT 是 NP 完全问题。对 SAT 的任意输入布尔表达式 φ，扫描 φ 找出两个**在 φ 中从未出现过的新布尔变量 u 和 v**，构造新布尔表达式：

$$\psi = f(\varphi) = \varphi \wedge (u \vee \neg u) \wedge (v \vee \neg v).$$

（注：必须把 $(u\vee\neg u)\wedge(v\vee\neg v)$ 显式写进 ψ 中，这样 u,v 才真正成为 ψ 的变量 $\mathrm{var}(\psi)=\mathrm{var}(\varphi)\cup\{u,v\}$！）

3. **第三步：双向证明** $\varphi \in \mathrm{SAT} \iff \psi \in$ **4TA-SAT**。
   - **正向（$\Rightarrow$）**：若 $\varphi\in\mathrm{SAT}$，则存在 $\mathrm{var}(\varphi)$ 上的赋值 τ 使 $\tau(\varphi)=1$。将 τ 扩展到新变量 (u,v) 上，分别取 $(u,v)\in\{(0,0),(0,1),(1,0),(1,1)\}$，由于 $(u\vee\neg u)$ 和 $(v\vee\neg v)$ 恒为 1，这 4 个扩展赋值全都令 $\psi=1$，且它们在 (u,v) 上取值各不相同，故 ψ 至少有 4 个不同的可满足赋值，即 $\psi\in$ 4TA-SAT。
   - **反向（$\Leftarrow$）**：若 $\psi\in$ 4TA-SAT，则 ψ 至少有 4 个可满足赋值，随便取出其中 1 个满足赋值 τ'。因为 $\psi=\varphi\wedge(u\vee\neg u)\wedge(v\vee\neg v)$ 为合取式，$\tau'(\psi)=1$ 迫使 $\tau'(\varphi)=1$。将 τ' 限制在原变量集 $\mathrm{var}(\varphi)$ 上，即得到 φ 的一个可满足赋值，故 $\varphi\in\mathrm{SAT}$。

4. **第四步：多项式时间分析**。构造 ψ 只需在 φ 末尾追加常数个字符，耗时 $O(|\varphi|)$。故 4TA-SAT 是 NP 完全问题。 □

10.9　题 9：证明半团问题（Half-Clique）是 NP 完全问题

题目定义：HALF-CLIQUE $=\{\langle G\rangle \mid$ 图$G=(V,E)$ 中存在一个恰好由$\frac{1}{2}|V|$ 个顶点组成的团$\}$。

1. **第一步：证明 HALF-CLIQUE** $\in$ NP。给定图 $G=(V,E)$，证书 c 为一个顶点子集 $C\subseteq V$。验证器在多项式时间内检查：（1）$|V|$ 为偶数且 $|C|=|V|/2$；（2）C 中任意两个不同顶点之间都有边相连。因此 HALF-CLIQUE $\in$ NP。

2. **第二步：构造归约** CLIQUE $\le_p$ **HALF-CLIQUE**。已知 CLIQUE $=\{\langle G,k\rangle \mid G$ 有大小至少为k 的团$\}$ 是 NP 完全问题。（先观察：如果一个图有大小 $\ge k$ 的团，从中任取 k 个点就是大小**恰好**为 k 的团。）
设输入为 $\langle G,k\rangle$，其中 $G=(V,E)$，顶点数 $n=|V|$。（若 $k\le 0$ 或 $k>n$，可直接算出真假并输出固定的二点连边图 K_2 或四孤立点图 $\overline{K_4}$；下面设 $1\le k\le n$。）

> **【直观理解】怎样把“原图有 k-团”变成“新图有恰好一半顶点的团”？** 原图有 n 个点，想找 k 个点的团；如果我们往图里新加两批辅助点：（1）加 n 个“全能万能点” U（它们自己内部两两连边，而且跟原图 V 的所有点也都连边，随时可以免费塞进团里凑人数）；（2）再加 $2k$ 个“完全孤立点” W（谁都不连，纯粹用来拉高分母总人数，绝不可能进团）。
> 看看现在的账本：新图 H 的总顶点数是 $n+n+2k=2n+2k$，它的一**半**刚好是 $\frac{2n+2k}{2}=n+k$！而孤立点 W 一个都进不了团，“全能点” U 最多只能提供 n 个点，剩下的 k 个名额**逼着你必须去原图 G 里找一个大小为 k 的团**！

据此正式写出新图 $H=(V',E')$ 的构造：

- **顶点集** V'：在原顶点集 V（n 个点）基础上，新增一个含 n 个顶点的集合 $U=\{u_1,\dots,u_n\}$，以及一个含 $2k$ 个顶点的集合 $W=\{w_1,\dots,w_{2k}\}$，总顶点数 $|V'|=n+n+2k=2(n+k)$（必为偶数，半团目标大小为 $n+k$）。
- **边集** E'：（1）保留原图 G 的所有边 E；（2）将 U 内部所有顶点对两两连边（使 U 成为 n-团）；（3）将 U 中每个顶点与原图 V 中每个顶点全部连边；（4）W 中的 $2k$ 个顶点不连任何边（保持为孤立点）。

3. **第三步：双向证明** $\langle G,k\rangle\in$ CLIQUE $\iff \langle H\rangle\in$ **HALF-CLIQUE**。
   - **正向（$\Rightarrow$）**：若 G 中存在大小至少为 k 的团，从中任取 k 个顶点组成 k-团 $C_0\subseteq V$。考察新图 H 中的顶点集 $C'=C_0\cup U$：它的大小为 $|C'|=k+n=\frac{1}{2}|V'|$；由于 C_0 内部两两有边、U 内部两两

有边、且 U 与 C_0 之间也全部有边，故 C' 是 H 的一个大小恰好为一半顶点数（$n+k$）的团，即 $\langle H\rangle \in$ HALF-CLIQUE。

- **反向（$\Leftarrow$）**：若 H 中存在一个大小为 $\frac{1}{2}|V'| = n+k$ 的半团 $C' \subseteq V'$。因为 $n \geq 1, k \geq 1$，团的大小 $n+k \geq 2$，所以团里每个顶点都至少要有 1 条边，故**孤立点集 W 中的任何顶点都不可能属于 C'**（即 $C' \subseteq V \cup U$）。又因为 U 一共只有 $|U| = n$ 个顶点，C' 最多只能包含 n 个来自 U 的顶点，因此 C' 中落在原图 V 里的顶点个数满足：

$$|C' \cap V| = |C'| - |C' \cap U| \geq (n+k) - n = k.$$

因为 $C' \cap V$ 作为团 C' 的子集，在原图 G 中两两相邻，所以 $C' \cap V$ 就是原图 G 中大小至少为 k 的团，即 $\langle G, k\rangle \in$ CLIQUE!

4. **第四步：多项式时间分析**。新图 H 的顶点数为 $2n+2k \leq 4n = O(n)$，边数至多为 $O(n^2)$，可在 $O(n^2)$ 多项式时间内构造完成。综上，半团问题是 NP 完全问题。 □

11　原课件速查索引与考前自测 18 问

11.1　遇到不懂的概念，去原课件哪一页查？

课件文件名	PDF 页码	涵盖的核心内容
`00-intro-slides.pdf`	第 1–5 页	课程目标、位值制 vs. 罗马数字编码、大整数乘法复杂度、教学路线
`07-图灵机.pdf`	第 1–2 页	确定图灵机七元组、瞬时描述（ID）、语言识别、单停机状态变形
`07-图灵机.pdf`	第 3–4 页	多带 TM（平方代价）、非确定 TM（指数代价）、文法、递归函数、TM 编码与通用图灵机 M_u
`08-图灵机和可计算.pdf`	第 1–3 页	可判定（递归）vs. 可识别（RE）、枚举图灵机、并交补连接闭包封闭性、基数不可数论证
`08-图灵机和可计算.pdf`	第 4–7 页	对角化语言 L_d、接受问题 L_{ACC}、停机问题 L_{HALT}、映射归约 $\leq_m$、$L_{\mathrm{ACC01}}/L_e/L_{\mathrm{REG}}/L_{\mathrm{EQ}}$、图灵归约 $\leq_T$
`08-图灵机和可计算.pdf`	第 8–10 页	Rice 定理及 8 个判断例题、波斯特对应问题（MPCP/PCP）、王氏贴砖问题
`12-计算复杂性.pdf`	第 1–4 页	时间与空间度量、DTIME/NTIME/DSPACE/NSPACE、P 与 NP、验证器、多项式归约 $\leq_p$、NPC 定义
`12-计算复杂性.pdf`	第 5–9 页	Cook 定理、CNF-SAT、3SAT、MAX-SAT、NAE-SAT、独立集 IS、团 CLIQUE、顶点覆盖 VC、支配集 DS、哈密顿与 TSP
`12-计算复杂性.pdf`	第 10–12 页	3DM、SUBSET-SUM、PARTITION、MPS、PSPACE 与 Savitch 定理、QBF、公式博弈 FG、广义地理博弈 GG、P 完全（L_{CFE}）、L 与 NL（PATH）
`12-计算复杂性.pdf`	第 13–15 页	MAX-SAT/IS/DS 归约详细证明页、循环有序三元组定义与图示

11.2　考前最后自测 18 问（左边遮住右边，能脱口而出就稳了）

自测问题	**核心答案要点**
1. 在 ID $01q10$ 中，带头正指着哪个字符？若 $\delta(q,1)=(p,X,L)$，下一步 ID 是什么？	指着第二个 1（状态右边的首字符）；左移后状态跨到左邻字符前面，变为 $0p1X0$。
2. 图灵机“识别 L”和“判定 L”到底差在哪里？	判定器对**所有输入**都必须停机；识别器在 $w\notin L$ 时允许永远死循环不停机。
3. 为什么证 RE 语言对并封闭时必须“交替运行”，而不能先跑完第一台再跑第二台？	因为第一台识别器在非成员输入上可能死循环，串行会导致第二台永远得不到执行机会。
4. 为什么 L 和 $\overline{L}$ 都可识别就能推出 L 可判定？	交替模拟两者的识别器；因为任意 w 必属于 L 或 $\overline{L}$ 之一，必有一台在有限步进入**接受**状态。
5. 要证目标问题 B 不可判定，映射归约方向该写 $A_{\mathrm{TM}}\leq_m B$ 还是 $B\leq_m A_{\mathrm{TM}}$？	必须写 $A_{\mathrm{TM}}\leq_m B$（已知难问题 $\leq_m$ 待证目标问题），利用“若 B 可解则 A_{TM} 可解”导出矛盾。
6. 构造辅助机器 N 时，为什么转换函数 f 不会因为 $M(w)$ 死循环而卡死？	因为 f 只负责把 $\langle M,w\rangle$ 当作静态文本拼接进 N 的源码里（只写代码，不运行 $M(w)$），必定停机。

自测问题	核心答案要点
7. 使用 Rice 定理必须检查哪两个条件？“机器至少有 37 个状态”能用 Rice 吗？	（1）只依赖语言 $L(M)$；（2）非平凡（有 RE 语言满足，也有 RE 语言不满足）。状态数看的是机器硬件而非语言，不能用 Rice（且状态数问题可判定）。
8. A_{TM}、$\overline{A_{\mathrm{TM}}}$、$L_e$（空语言）、$L_{\mathrm{EQ}}$（语言相等）分别属于哪一层？	A_{TM} 可识别但不可判定；$\overline{A_{\mathrm{TM}}}$ 和 L_e 不可识别（但 $\overline{L_e}$ 可识别）；L_{EQ} 与 $\overline{L_{\mathrm{EQ}}}$ 均不可识别。
9. 什么是 NP 的验证器定义？为什么“枚举所有证书逐一检验”不能证明 P = NP？	$x \in L \iff \exists c\ (|c| \le \mathrm{poly}(|x|))$ 使多项式时间验证器 $V(x,c)=1$。因为长 n^k 的候选证书有 2^{n^k} 个，穷举需要指数时间。
10. 把单文字子句 z 和二文字子句 $(x \vee y)$ 转成 3CNF 分别要拆成几个子句？	单文字 z 引入 u, v 拆成 4 个子句；二文字 $(x \vee y)$ 引入 u 拆成 2 个子句。
11. 3SAT $\le_p$ IS 构造中，三角形内部边和互补文字边分别起什么作用？	三角形边迫使每个子句至多选 1 个点（凑够 m 个点就必须每子句选 1 个）；互补边禁止同时选 x 和 $\neg x$（保证赋值无矛盾）。
12. IS $\le_p$ CLIQUE 和 IS $\le_p$ VC 在图和阈值 k 的变换上有什么不同？	转团：取补图 $\overline{G}$，阈值仍为 k；转顶点覆盖：保持原图 G 不变，顶点集取补集 $V \setminus I$，阈值变为 $\lvert V\rvert - k$。
13. 哈密顿环归约到 TSP 时，为什么原边权设为 1、非原边权设为 2、预算设为 n？	因为巡回圈必含 n 条边，每条边权 ≥ 1；总权 $\le n$ 强迫所选的 n 条边每条权重都等于 1（即全为原图边）。
14. 子集和问题有 $O(nt)$ 动态规划算法，为什么它仍然是 NP 完全问题？	因为目标和 t 在输入中用二进制编码，长度只有 $m = \log_2 t$；$O(nt) = O(n \cdot 2^m)$ 相对于输入长度是指数级（伪多项式时间）。
15. 为什么 PSPACE = NPSPACE（Savitch 定理），空间怎么做到比时间省？	时间不可复用，但工作带空间擦除后可反复重用；用中点二分递归检查 2^t 步可达性，递归深度 $O(S(n))$，每层空间 $O(S(n))$，总空间 $O(S(n)^2)$。
16. 为什么定义 P 完全和 NL 完全时，要用对数空间归约 $\le_L$ 而不用 $\le_p$？	因为若允许归约函数自己跑多项式时间，归约函数自己就能直接把 P 或 NL 里的原问题解出来，导致归约失去区分度。
17. 设计单带图灵机识别 $\{ww\}$ 与 $\{ww^R\}$ 的本质区别是什么？	$\{ww^R\}$ 直接比对首尾即可；$\{ww\}$ 不能直接比首尾，必须先双向夹逼标出中点分成 L 区和 R 区，再同向逐位比对。
18. 作业题 9（半团）为什么要在加 n 个全连接点 U 之外，再加 $2k$ 个孤立点 W？	孤立点绝不能进团，只用来把总顶点数凑成 $2n + 2k$（半团目标 $n + k$）；扣除 U 最多贡献的 n 个点后，迫使原图必须提供 k-团。